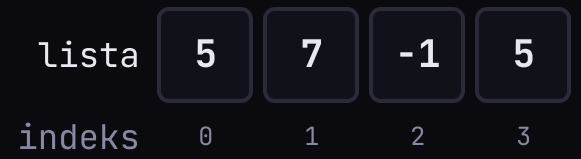
// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Vector w C++ — krok po kroku

Wszystko o vectorze, powoli i po kolei: tworzenie, indeksowanie, metody, wypisywanie, pętla for-each z referencją, iteratory begin() i end(), wczytywanie i vector 2D. Potem podtablice, podzbiory, prefiksy i sufiksy — a na końcu zadania z notatek, część z rozwiązaniem krok po kroku.

C++ · LEKCJA 10 · VECTOR · PO TABLICACH, STRINGACH I PARACH



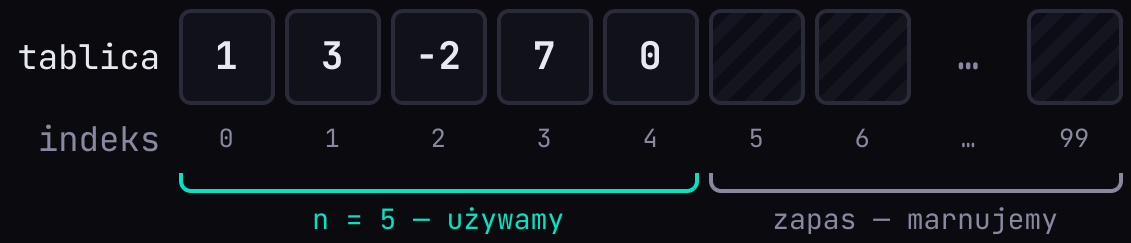
PLAN PREZENTACJI

01 Czym jest vector?
02 Tworzenie vectora
03 Indeksowanie
04 size, push_back, clear
05 Więcej metod
06 Wypisywanie vectora
07 Pętla for-each i &
08 Iteratory: begin i end
09 Wczytywanie vectora
10 Vector 2D
11 Podtablica
12 Podzbiór
13 Prefiks i sufiks
14 Zadania
15 Najczęstsze błędy
16 Podsumowanie

01 Tablica ma pewien problem

LEKCJA 7 — TABLICA Z ZAPASEM

```
int tablica[100]; // 100 >= n
int n;
cin >> n;
```



Pamiętasz najważniejszą zasadę tablic? **Rozmiar ustalamy na początku i nie da się go zmienić.**

1. Musimy z góry znać **największe n** i zrobić tablicę z zapasem.
2. Często **marnujemy** miejsce — używamy 5 szufladek ze 100.
3. Musimy osobno pamiętać `n` — tablica sama nie wie, ile liczb w niej naprawdę jest.
4. Nie da się **dopisać** nowej liczby na koniec.

Na ratunek: **vector** rozwiązuje wszystkie te problemy.

01 Vector = tablica, która umie rosnąć

vector to lista liczb (albo znaków, napisów…) — jak tablica, ale **jej rozmiar może się zmieniać** w trakcie programu. Sama też pamięta, ile ma elementów.

	Tablica (lekcja 7)	vector (dzisiaj)
rozmiar	stały — ustalony na początku	**może rosnąć i maleć**
ile elementów?	trzeba pamiętać osobno w `n`	mówi sam: `lista.size()`
dodanie na koniec	nie da się	`lista.push_back(5);`
indeksy	od 0 do n-1	tak samo: od 0 do n-1
odczyt / zmiana	`t[2]`, `t[2] = 7;`	tak samo: `lista[2]`, `lista[2] = 7;`
wypisanie jednym cout	nie	też nie — pętlą

01 Szablon programu z vectorem

SZABLON KAŻDEGO PROGRAMU

```
#include <iostream>    // cin i cout
#include <vector>      // tu mieszka vector
using namespace std;   // vector zamiast std::vector

int main(){
    // <- tu wpisujemy kod z vectorami
}
```

vector jest w bibliotece **vector**. Dopisujemy ją na górze, obok iostream.

> Bez `#include <vector>` program się nie skompiluje — kompilator nie będzie wiedział, co to jest vector.

Na kolejnych slajdach często pokazujemy tylko **środek main** — reszta jest zawsze taka sama.

02 Budowa linijki, która tworzy vector

Linijka z notatek, rozłożona na części:

```
vector<int> lista;
```

1 2 3 4

1	`vector`	„to będzie **vector**” — lista, która może rosnąć
2	`<int>`	w nawiasach ostrych: **jakiego typu** są elementy (tu: liczby całkowite)
3	`lista`	**nazwa zmiennej** — wymyślamy ją sami
4	`;`	**średnik** na końcu

> Tak jak przy parze: `vector<int>` to razem **typ**. W nawiasach ostrych piszemy, **co przechowuje** nasza struktura.

02 Sposób 1: pusty vector

Z NOTATEK

```
vector<int> lista;
```

Nie podaliśmy ani rozmiaru, ani wartości. Taki vector jest **pusty** — ma **0 elementów**.



Globalnie (poza main)

lista = **{}** — pusty

Lokalnie (w main)

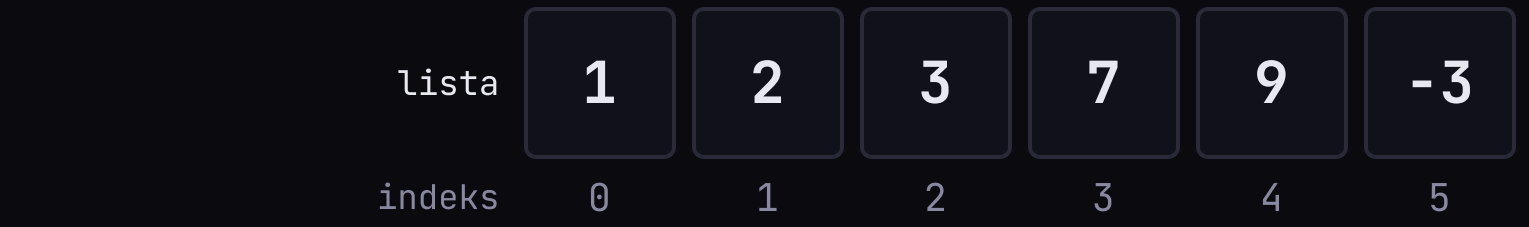
lista = **{}** — też pusty

Inaczej niż w tablicy: tu **nie ma śmieci**, bo nie ma w ogóle żadnych szufladek. Elementy dodamy później (rozdział 04).

02 Sposób 2: od razu z wartościami

Z NOTATEK

```
vector<int> lista = {1, 2, 3, 7, 9, -3};
```



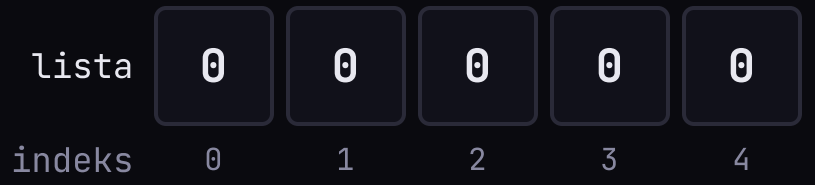
Po znaku **=** w klamrach podajemy wszystkie elementy po kolei — dokładnie jak w liście inicjalizacyjnej tablicy. Rozmiaru **nie piszemy** — vector sam policzy, że ma 6 elementów.

02 Sposób 3: n razy ta sama liczba

```
vector<int> lista(5, 0);
            ①  ②
```

① 5 — **ile razy** powtórzyć element

② 0 — **dokładny element**, który powtarzamy



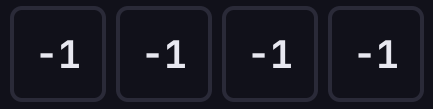
W nawiasach **okrągłych** podajemy dwie liczby: najpierw **ile**, potem **co**. Czyli: „pięć razy zero”.

02 Sposób 3 — więcej przykładów

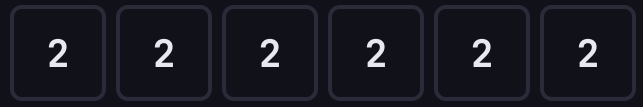
```
vector<int> a(3, 7);
```
trzy razy siódemka




```
vector<int> b(4, -1);
```
cztery razy minus jeden




```
vector<int> c(6, 2);
```
sześć dwójek




```
vector<char> d(5, 'x');
```
pięć razy znak x



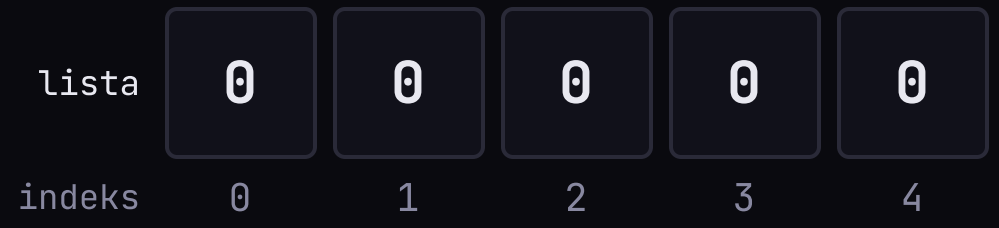

```
vector<string> e(3, "C++");
```
trzy napisy C++



02 Sposób 4: n zer

Z NOTATEK

```
vector<int> lista(5);
```



Podajemy tylko **ile** elementów. Drugą liczbę pomijamy — wtedy C++ wstawia **wartość domyślną**: dla int to **0**.

W notatkach to „default constructor" — konstruktor domyślny. Wynik: **{0, 0, 0, 0, 0}**.

Czyli: `vector<int> lista(5);` daje to samo co `vector<int> lista(5, 0);`

Zera są **zawsze** — globalnie i lokalnie. Śmieci w vectorze nie ma.

02 Cztery sposoby — porównanie

KOD Z NOTATEK

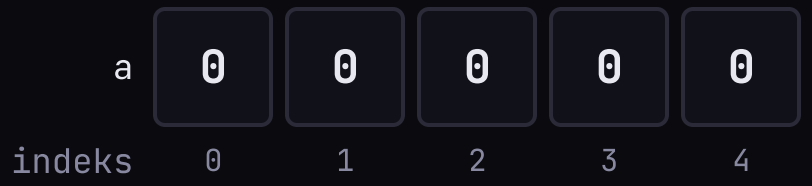
```
vector<int> lista;                      // {}
vector<int> lista1 = {1, 2, 3, 7, 9, -3};  // {1, 2, 3, 7, 9, -3}
vector<int> lista2(5, 0);               // {0, 0, 0, 0, 0}
vector<int> lista3(5);                  // {0, 0, 0, 0, 0}
```

Zapis	Co robi	Rozmiar
`vector<int> lista;`	pusty vector	0
`= {1, 2, 3, 7, 9, -3}`	dokładnie te elementy	6
`(5, 0)`	pięć razy 0 — **ile, co**	5
`(5)`	pięć wartości domyślnych (0)	5

02 Uwaga: nawiasy okrągłe a klamry

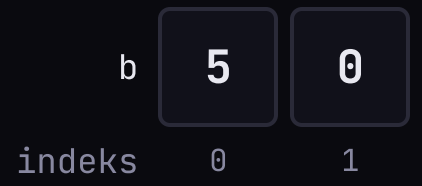
NAWIASY OKRĄGŁE: ILE, CO

```
vector<int> a(5, 0);
```



KLAMRY: DOKŁADNE WARTOŚCI

```
vector<int> b = {5, 0};
```



Uwaga: te dwie linijki wyglądają podobnie, a robią coś **zupełnie innego**! a ma **pięć zer**, a b ma **dwa elementy**: 5 i 0.

02 Vector może trzymać dowolny typ

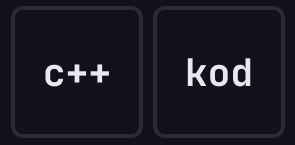
```
vector<char> znaki = {'a', 'b', 'c'};
```

znaki



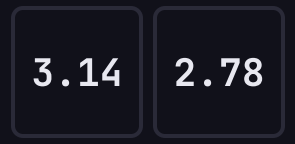

```
vector<string> slowa = {"c++", "kod"};
```

napisy (lekcja 8)




```
vector<double> ulamki = {3.14, 2.78};
```

liczby z przecinkiem




```
vector<pair<int, int>> pary = {{0, 0}, {1, 5}};
```

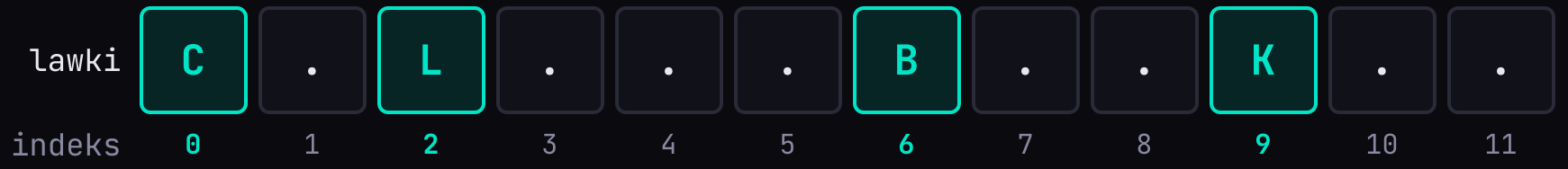
pary (lekcja 9)



Na lekcji o parach przykłady z notatek były z vectorem par — teraz możesz je już napisać tak, jak w notatkach: `vector<pair<int, int>> lista;`.

03 Indeksowanie = odczytywanie elementu na pewnej pozycji

W parku stoi rząd **12 ławek**. Na niektórych ktoś siedzi: C, L, B i K. Kropka oznacza pustą ławkę. Każda ława ma swój numer — **indeks** — od 0 do 11.

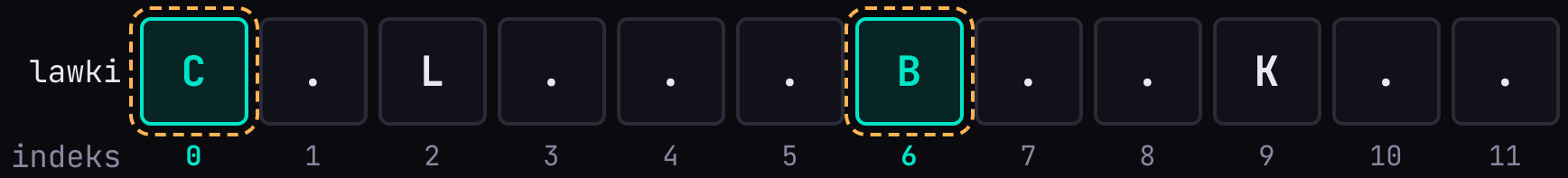


Indeksowanie w vectorze działa **dokładnie tak samo** jak w tablicy: indeksy od **0** do **n−1**, nazwa, a za nią indeks w nawiasach kwadratowych.

03 Ławki w kodzie

Z NOTATEK

```
vector<char> lawki = {'C', '.', 'L', '.', '.', '.', 'B', '.', '.', 'K', '.', '.'};
```



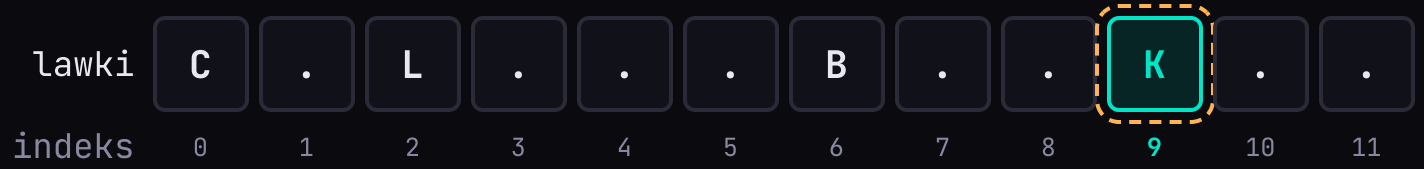

```
cout << lawki[0] << endl;   // wyświetla C
cout << lawki[6] << endl;   // wyświetla B
```

NA EKRANIE

```
C
B
```

vector<char> — każda szufladka trzyma **jeden znak**, zapisany w apostrofach: 'C', '.'. W nazwach zmiennych nie używamy polskich liter — dlatego lawki, a nie ławki.

03 Sprawdzamy, kto siedzi na ławce 9



Z NOTATEK

```
if(lawki[9] == 'K'){
    cout << "na indeksie 9 jest litera K" << endl;
}
```

NA EKRANIE

```
na indeksie 9 jest litera K
```

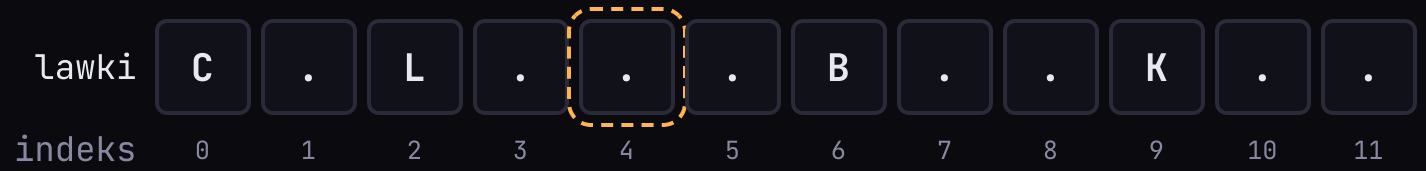
1. `lawki[9]` — bierzemy znak z ławki o indeksie 9: **K**.
2. Porównujemy go ze znakiem `'K'` — dwoma znakami równości `==`.
3. Są równe → warunek jest prawdziwy → wypisujemy napis.

03 Modyfikowanie: ktoś siada na ławce 4

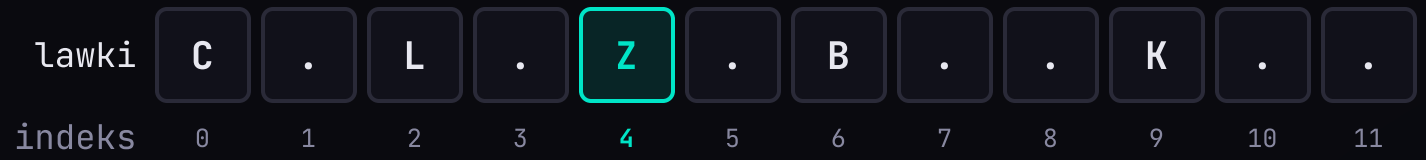
Z NOTATEK

```
lawki[4] = 'Z';
```

przed



po



Do elementu vectora wpisujemy nową wartość znakiem **=** — tak samo jak w tablicy. Na ławce 4 siedzi teraz Z, reszta ławek się nie zmienia.

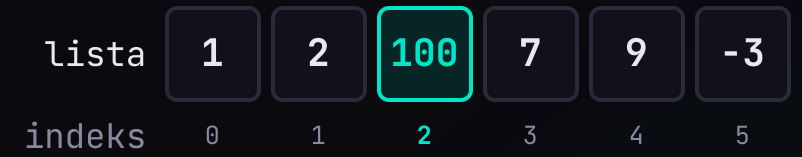
03 Indeksowanie liczb — więcej przykładów

```cpp
vector<int> lista = {1, 2, 3, 7, 9, -3};

cout << lista[0] << endl;              // 1
cout << lista[5] << endl;              // -3
cout << lista[1] + lista[3] << endl;   // 2 + 7
lista[2] = 100;
cout << lista[2] << endl;              // 100
```

NA EKRANIE

```
1
-3
9
100
```



03 Uwaga: indeks poza vectorem

ŹLE

```
vector<int> lista = {1, 2, 3, 7, 9, -3};
cout << lista[6] << endl;   // BŁĄD
```

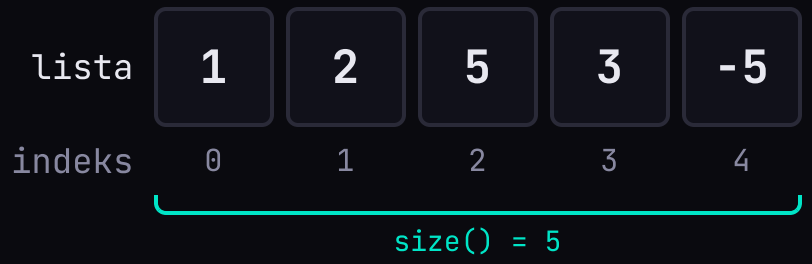


Uwaga: tak jak w tablicy: vector o 6 elementach ma indeksy **0…5**. lista[6] to pamięć **za vectorem** — program wypisze śmieci albo się wysypie.

04 lista.size() — ile jest elementów

Z NOTATEK: SIZE() ZWRACA WIELKOŚĆ

```
vector<int> lista = {1, 2, 5, 3, -5};
cout << lista.size() << endl;
```



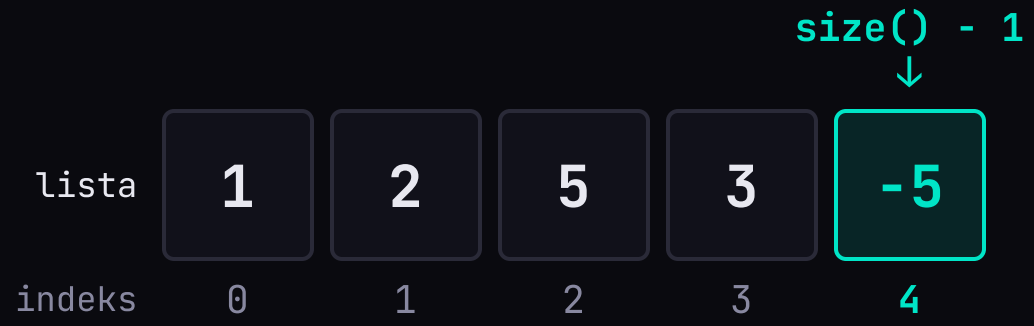
size() to **metoda** vectora — polecenie, które piszemy po kropce, z nawiasami na końcu. Zwraca liczbę elementów.

NA EKRANIE

```
5
```

04 Ostatni element: lista[lista.size() - 1]

```
vector<int> lista = {1, 2, 5, 3, -5};
cout << lista[lista.size() - 1] << endl;   // -5
```



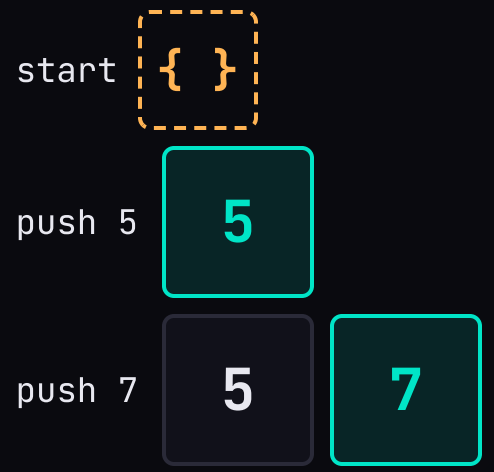
1. lista.size() = **5**.
2. 5 − 1 = **4** — to indeks ostatniego elementu (n−1, jak w tablicy).
3. lista[4] = **−5**.

04 push_back — dodaje element na koniec

Z NOTATEK

```
vector<int> lista;
// lista = {}
lista.push_back(5);
// lista = {5}
lista.push_back(7);
// lista = {5, 7}
```

push_back **dodaje nowy element z prawej strony** — na tył vectora. Vector wydłuża się o 1.



04 push_back krok po kroku — size rośnie

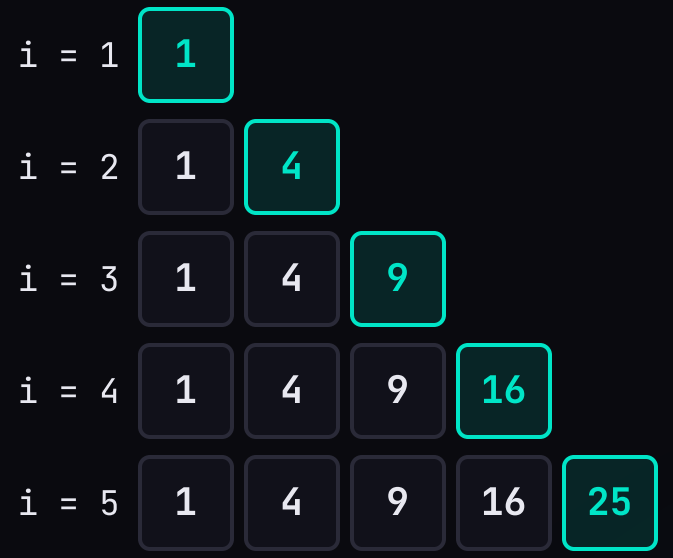
Linijka	lista po tej linijce	lista.size()
`vector<int> lista;`	{ }	0
`lista.push_back(5);`	{5}	1
`lista.push_back(7);`	{5, 7}	2
`lista.push_back(-1);`	{5, 7, -1}	3
`lista.push_back(5);`	{5, 7, -1, 5}	4

Każdy push_back dokłada **jeden** element na sam koniec i zwiększa size() o 1. Stare elementy się nie ruszają.

04 push_back w pętli

```
vector<int> kwadraty;
for(int i = 1; i <= 5; i++){
    kwadraty.push_back(i * i);
}
```

Startujemy od pustego vectora. W każdym obrocie pętli dokładamy na koniec kwadrat liczby i.



04 Pusty vector nie ma szufladek

ŹLE

```
vector<int> lista;
lista[0] = 5;    // BŁĄD!
```

DOBRZE

```
vector<int> lista;
lista.push_back(5);    // dobrze
```



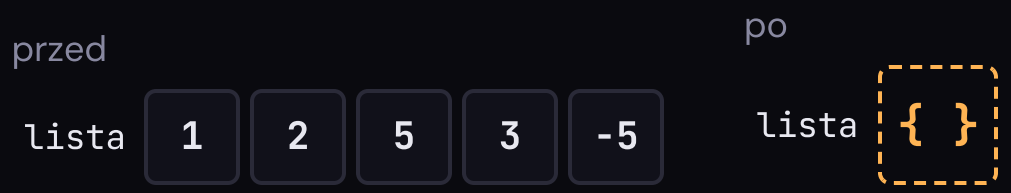
Uwaga: w pustym vectorze **nie ma** szufladki o indeksie 0. Indeks **zmienia istniejący** element, a push_back **tworzy nowy**.

Gdy chcesz od razu mieć n szufladek, stwórz `vector<int> lista(n);` — wtedy lista[0] istnieje.

04 clear — usuwa wszystkie elementy

Z NOTATEK: CLEAR() USUWA WSZYSTKIE ELEMENTY

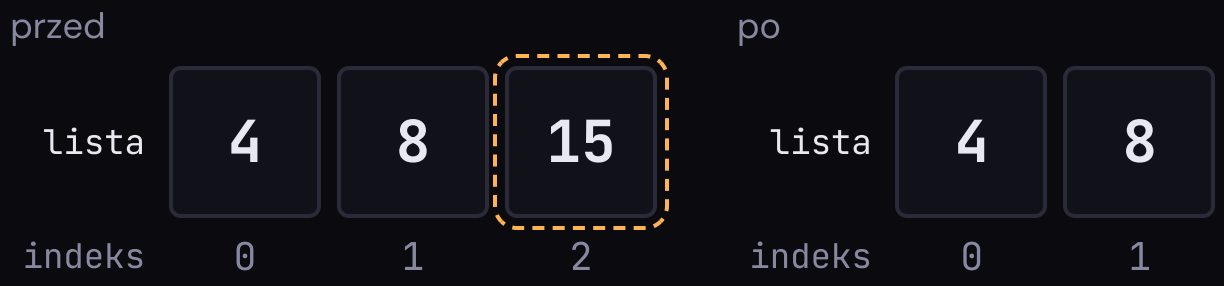
```
vector<int> lista = {1, 2, 5, 3, -5};
lista.clear();
```



Po clear() vector jest pusty: size() = 0. Można go dalej używać — np. znowu robić push_back.

05 pop_back — usuwa ostatni element

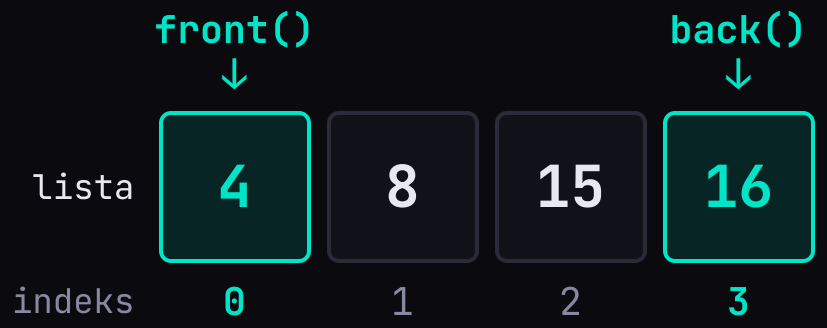
```
vector<int> lista = {4, 8, 15};
lista.pop_back();
```



pop_back to przeciwieństwo push_back: zabiera **ostatni** element z prawej strony. size() maleje o 1.

05 front i back — pierwszy i ostatni

```
vector<int> lista = {4, 8, 15, 16};
cout << lista.front() << endl;   // 4
cout << lista.back() << endl;    // 16
```



Metoda	To samo co
`lista.front()`	`lista[0]`
`lista.back()`	`lista[lista.size() - 1]`

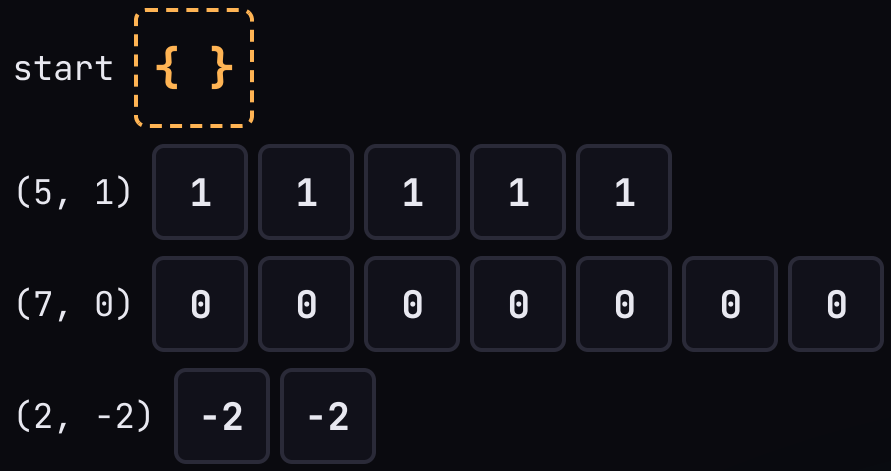
NA EKRANIE

```
4
16
```

05 assign — wypełnij od nowa

Z NOTATEK

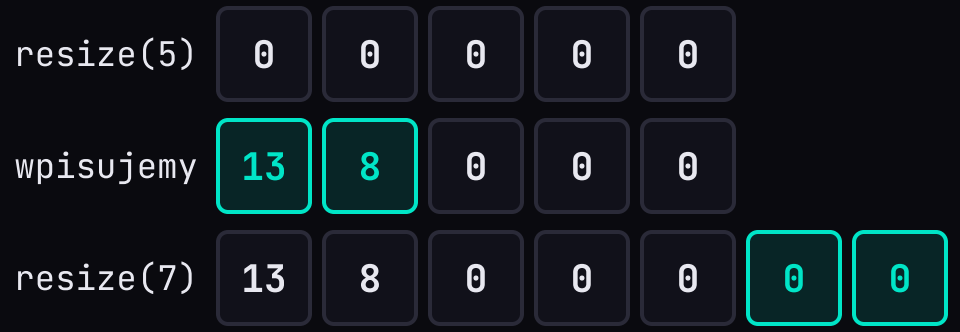
```
vector<int> lista;     // pusty: {}
lista.assign(5, 1);    // {1,1,1,1,1}
lista.assign(7, 0);    // {0,0,0,0,0,0,0}
lista.assign(2, -2);   // {-2,-2}
```



assign(ile, co) **wyrzuca całą starą zawartość** i wpisuje **ile** razy element **co**. Z notatek: „tak samo można zrobić z konstruktorem” — `vector<int> lista(5, 1);`.

05 resize — wydłużanie

```
vector<int> lista;    // {}
lista.resize(5);      // {0, 0, 0, 0, 0}
lista[0] = 13;
lista[1] = 8;         // {13, 8, 0, 0, 0}
lista.resize(7);      // {13, 8, 0, 0, 0, 0, 0}
```



resize(n) zmienia liczbę elementów na **n**. Gdy n jest **większe**, vector dokłada na końcu nowe elementy z wartością domyślną — dla int to **0**. Stare elementy zostają na swoich miejscach.

05 resize — skracanie

```
// lista = {13, 8, 0, 0, 0, 0, 0}
lista.resize(2);      // {13, 8}
```



Gdy nowy rozmiar jest **mniejszy**, vector **obcina** elementy z końca. Zostają tylko pierwsze dwa.

06 Nie da się wypisać vectora jednym cout

TAK NIE ROBIMY

```
vector<int> lista = {1, 2, 5, 3, -5};
cout << lista << endl;   // BŁĄD
```

KOMPILATOR MÓWI

```
error: no match for 'operator<<'
```

Tak jak przy parze — program się nie skompiluje. Każdy element trzeba wypisać **osobno**, w pętli.

Nie da się wypisać vectora jednym cout.

> **Są** dwa sposoby z notatek: pętla po indeksach i pętla for-each.

06 Sposób 1: pętla po indeksach

Z NOTATEK

```
vector<int> lista = {1, 2, 5, 3, -5};
for(int i = 0; i < lista.size(); i++){
    cout << lista[i] << ' ';
}
cout << endl;
```

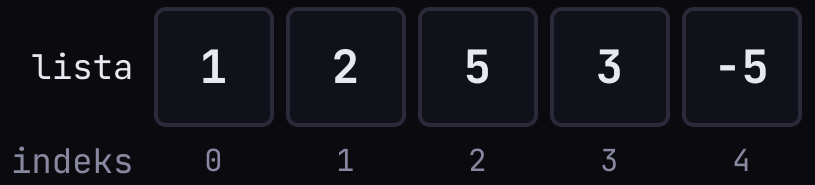
NA EKRANIE

```
1 2 5 3 -5
```

Dokładnie jak przy tablicy. Jedyna różnica: zamiast `n` piszemy `lista.size()` — vector sam wie, ile ma elementów.

> W notatkach w warunku jest `i < n` — wtedy `n` musi być równe liczbie elementów. Najpewniej jest napisać `i < lista.size()`.

06 Sposób 1 — krok po kroku



i	i < size() ?	wypisujemy	na ekranie po tym kroku
0	tak	`lista[0]` = 1	`1`
1	tak	`lista[1]` = 2	`1 2`
2	tak	`lista[2]` = 5	`1 2 5`
3	tak	`lista[3]` = 3	`1 2 5 3`
4	tak	`lista[4]` = -5	`1 2 5 3 -5`
5	**nie**	koniec pętli	`1 2 5 3 -5` + endl

06 Sposób 2: pętla for–each

Z NOTATEK

```
vector<int> lista = {1, 2, 5, 3, -5};
for(auto el : lista){
    cout << el << ' ';
}
cout << endl;
```

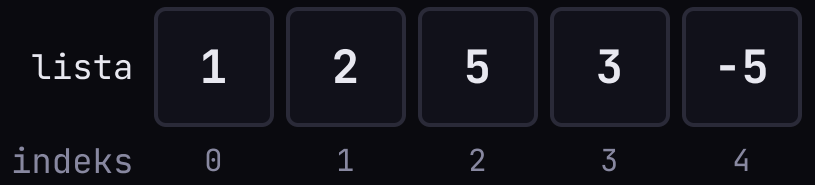
NA EKRANIE

```
1 2 5 3 -5
```

`for(auto el : lista)` czytamy: **„dla każdego elementu el z listy”**.

1. Pętla sama bierze elementy po kolei: najpierw 1, potem 2, 5, 3 i -5.
2. W każdym obrocie zmienna `el` **to kolejny element** — wartość, a nie indeks!
3. Nie piszemy żadnego i, warunku ani i++.

06 for-each — krok po kroku



Obrót	el	na ekranie po tym kroku
1	**1**	`1`
2	**2**	`1 2`
3	**5**	`1 2 5`
4	**3**	`1 2 5 3`
5	**-5**	`1 2 5 3 -5`

el przyjmuje **kolejne wartości** elementów. Gdy elementy się skończą, pętla sama się kończy.

06 auto — C++ sam zgaduje typ

`auto` znaczy: „C++, ustal typ sam”. W pętli for-each auto dostaje typ elementów vectora:

vector	pętla	el jest typu
`vector<int> lista`	`for(auto el : lista)`	`int`
`vector<char> lawki`	`for(auto el : lawki)`	`char`
`vector<string> slowa`	`for(auto el : slowa)`	`string`
`vector<pair<int, int>> pary`	`for(auto el : pary)`	`pair<int, int>`

Można też napisać typ samemu: `for(int el : lista)` — działa tak samo. auto jest po prostu wygodniejsze.

06 Który sposób wybrać?

Dla `vector<int> lista = {1, 2, 5};`:

POTRZEBNY INDEKS → PĘTLA PO INDEKSACH

```
for(int i = 0; i < lista.size(); i++){
    cout << i << ": " << lista[i] << endl;
}
```

NA EKRANIE

```
0: 1
1: 2
2: 5
```

SAME WARTOŚCI → FOR-EACH

```
for(auto el : lista){
    cout << el << ' ';
}
```

NA EKRANIE

```
1 2 5
```

Gdy potrzebujesz **numeru** elementu (np. żeby wypisać indeks albo porównać sąsiadów) — pętla po indeksach. Gdy wystarczą **wartości** — for-each jest krótszy.

07 Dwie wersje pętli for–each

Z NOTATEK

```
// 1. zwykły for-each-loop
for(int e : lista){
    ...
}
// 2. for-each-loop z referencją
for(int &e : lista){
    ...
}
```

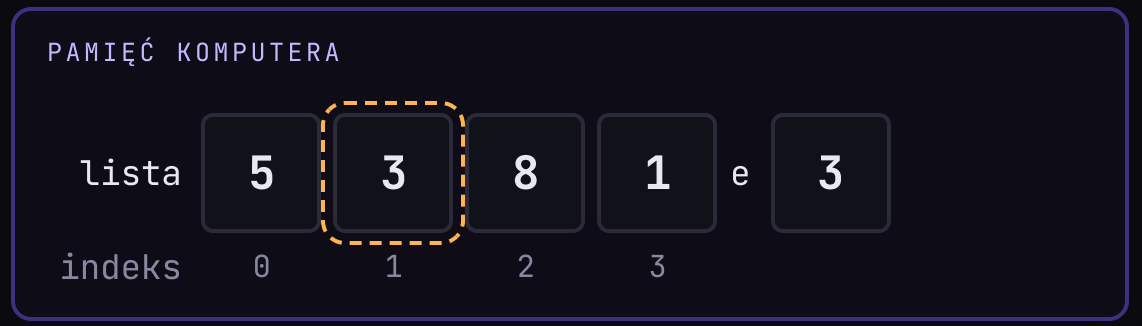
Różnią się tylko **jednym znakiem**: `&`.

> Ten znak to **referencja**. Wygląda niewinnie, ale zmienia bardzo dużo — zobaczmy krok po kroku, co dokładnie.

07 Wersja 1: e to kopia

```
for(int e : lista)
```

1. e przechowuje **taką samą wartość** jak element w liście...
2. ...ale zmienna e **nie jest** przechowywana w tym samym miejscu pamięci.
3. e to **kopia** — osobna szufladka obok listy.



Więc: jeżeli zmodyfikujemy e, to **nie zmodyfikuje się** lista.

07 Przykład: podwajamy bez &

```
vector<int> lista = {1, 2, 3};
for(int e : lista){
    e = e * 2;      // zmienia tylko kopię
}
for(auto el : lista){
    cout << el << ' ';
}
```

NA EKRANIE

```
1 2 3
```

Obrót	kopia e	e po e * 2	lista
1	1	2	{1, 2, 3}
2	2	4	{1, 2, 3}
3	3	6	{1, 2, 3}

Każda kopia e została podwojona — ale **lista się nie zmieniła**. Po każdym obrocie kopia znika.

07 Wersja 2: e to referencja

```
for(int &e : lista)
```

1. e jest **w tym samym miejscu pamięci**, w którym jest element listy.
2. e to po prostu **druga nazwa** tej samej szufladki.
3. Nie ma żadnej kopii.



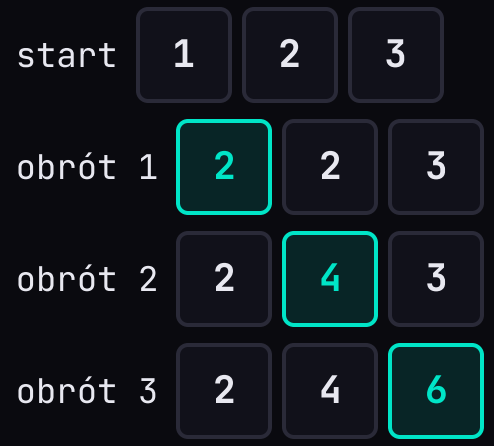
Więc: jeżeli zmodyfikujemy e, to **zmodyfikuje się** lista.

07 Przykład: podwajamy z &

```
vector<int> lista = {1, 2, 3};
for(int &e : lista){
    e = e * 2;      // zmienia element listy
}
for(auto el : lista){
    cout << el << ' ';
}
```

NA EKRANIE

```
2 4 6
```



Tym razem e to sama szufladka listy — więc lista naprawdę się zmienia.

07 Bez & czy z &?

	`for(int e : lista)`	`for(int &e : lista)`
czym jest e	kopią elementu	tym samym elementem (referencją)
miejsce w pamięci	osobne	to samo co element listy
zmiana e zmienia listę?	nie	tak
kiedy używać	gdy tylko czytamy / wypisujemy	gdy zmieniamy albo wczytujemy elementy

Z auto działa to tak samo: `for(auto e : lista)` to kopia, `for(auto &e : lista)` — referencja.

08 Po co nowe pojęcie?

Do tej pory miejsce w vectorze wskazywaliśmy **indeksem**: `lista[2]`. Ale niektóre polecenia **nie przyjmują indeksu**. Zamiast numeru chcą dostać coś innego:

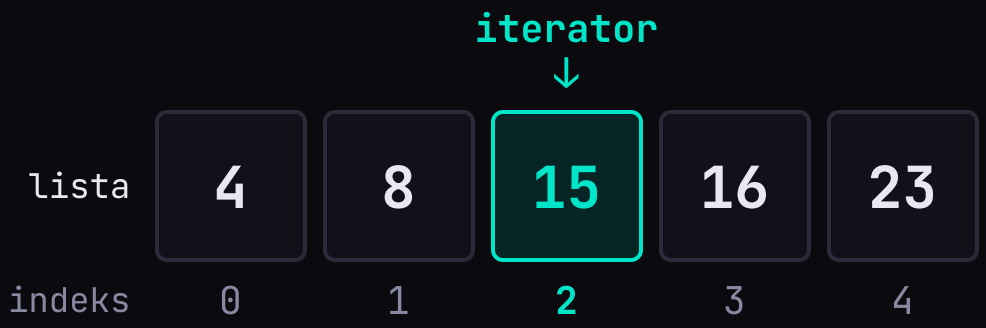
```
lista.erase(lista.begin() + 1);          // usuń element — za chwilę
lista.insert(lista.begin(), 7);          // wstaw element — za chwilę
sort(lista.begin(), lista.end());        // posortuj — następna lekcja
```

Te zaznaczone kawałki to **iteratory**. Zanim użyjemy erase i insert, musimy dokładnie zrozumieć trzy rzeczy:

1. **Co to jest** iterator?
2. Na co wskazuje `lista.begin()`, a na co `lista.end()`?
3. **Dlaczego** w ogóle używamy iteratorów, skoro mamy indeksy?

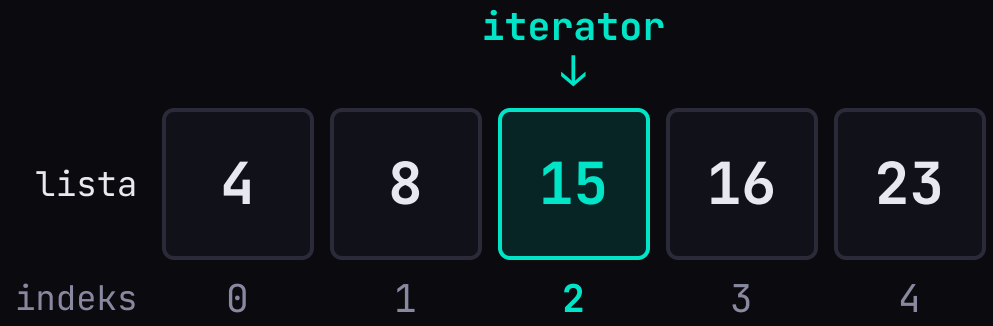
08 Co to jest iterator?

Iterator to strzałka, która pokazuje jeden element vectora.



1. Pomyśl o ławkach z parku: iterator to **palec**, którym pokazujesz jedną ławkę. Palec to nie osoba na ławce — on tylko mówi, **gdzie** patrzymy.
2. Iterator to **nie wartość** (15) i **nie numer** (2). To wskazanie miejsca w vectorze.
3. Strzałkę można **przesuwać** — o jeden element albo o kilka, w prawo albo w lewo.

08 Indeks, iterator, wartość — trzy różne rzeczy

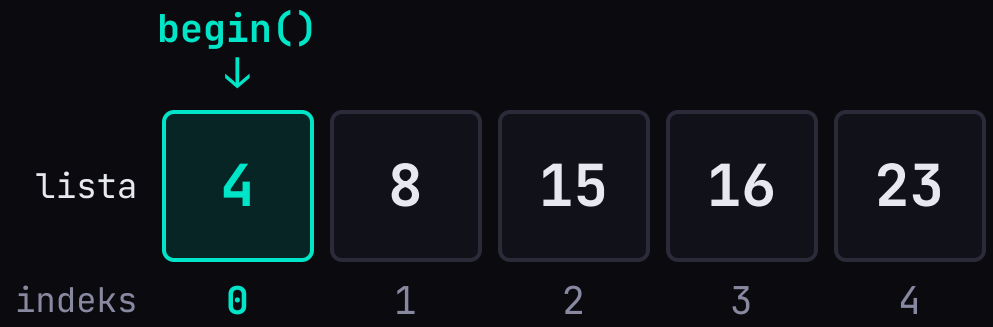


Pojęcie	Przykład	Co to jest
indeks	`2`	**numer szufladki — zwykła liczba int**
iterator	`lista.begin() + 2`	**strzałka pokazująca szufladkę**
wartość	`15`	**to, co leży w szufladce**

Skojarzenie z książką: **indeks** to numer strony, **iterator** to zakładka włożona między kartki, a **wartość** to tekst wydrukowany na tej stronie.

08 begin() — strzałka na pierwszy element

```
vector<int> lista = {4, 8, 15, 16, 23};
auto it = lista.begin();   // it pokazuje na 4
```



1. `begin()` to metoda vectora — tak jak `size()` — więc piszemy ją po kropce, z nawiasami.
2. Zwraca iterator, który pokazuje na **pierwszy** element — ten o indeksie 0.
3. Zapisujemy go do zmiennej `it`. Typ iteratora ma długą nazwę (`vector<int>::iterator`), dlatego piszemy `auto` — znamy je z pętli for-each.

08 Gwiazdka * — co leży pod strzałką?

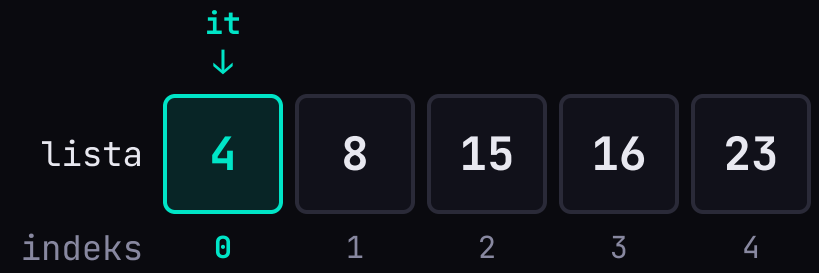
```
vector<int> lista = {4, 8, 15, 16, 23};
auto it = lista.begin();
cout << *it << endl;   // 4
```

NA EKRANIE

```
4
```

BEZ GWIAZDKI — BŁĄD KOMPILACJI

```
cout << it << endl;    // BŁĄD
```

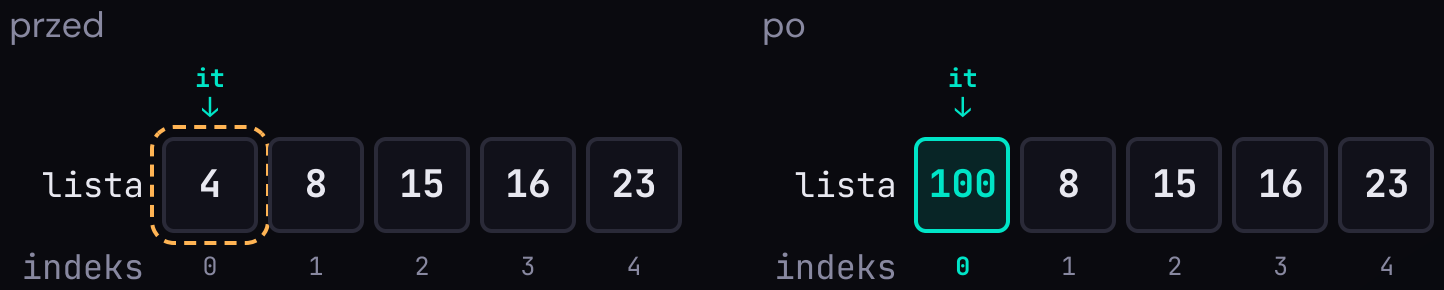


Zapis	Co to jest
`it`	strzałka (iterator)
`*it`	to, na co strzałka pokazuje — **4**

Gwiazdkę czytamy: „to, co jest pod strzałką”. Sam iterator to nie liczba — dlatego cout bez gwiazdki się nie skompiluje.

08 Przez *it można też zmienić element

```
vector<int> lista = {4, 8, 15, 16, 23};
auto it = lista.begin();
*it = 100;                  // zmieniamy to, co jest pod strzałką
```



*it to **ta sama szufladka**, na którą pokazuje strzałka — tak jak referencja **&** z pętli for-each. Zmiana *it zmienia vector.

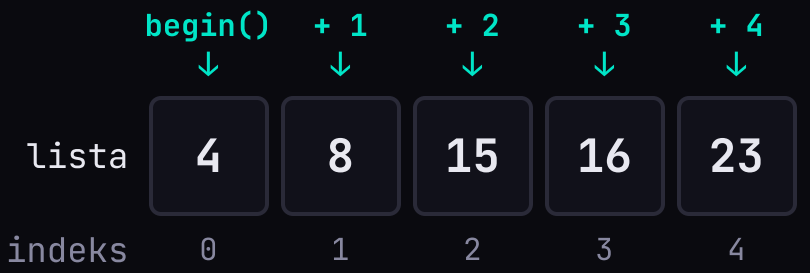
08 Przesuwanie strzałki: it++ i it + k

```
vector<int> lista = {4, 8, 15, 16, 23};
auto it = lista.begin();   // pokazuje na 4
it++;                      // pokazuje na 8
it = it + 2;               // pokazuje na 16
cout << *it << endl;       // 16
```



`it++` przesuwa strzałkę o **jeden** element w prawo, a `it + k` — o **k** elementów. W lewo tak samo: `it--` i `it - k`.

08 lista.begin() + k — strzałka na indeks k



Iterator	pokazuje na indeks	* daje wartość
`lista.begin()`	0	4
`lista.begin() + 1`	1	8
`lista.begin() + 2`	2	15
`lista.begin() + 3`	3	16
`lista.begin() + 4`	4	23

Zapamiętaj: `lista.begin() + k` pokazuje na element o indeksie k. Czyli `*(lista.begin() + k)` to to samo co `lista[k]`.

08 end() — strzałka ZA ostatnim elementem

```
auto koniec = lista.end();   // pokazuje ZA 23
```



Uwaga: `end()` **nie** pokazuje na ostatni element (23)! Pokazuje na miejsce **tuż za nim**, w którym nie ma już żadnego elementu — jak meta za ostatnim zawodnikiem albo puste miejsce za ostatnią osobą w kolejce.

08 Nie wolno zaglądać pod end()

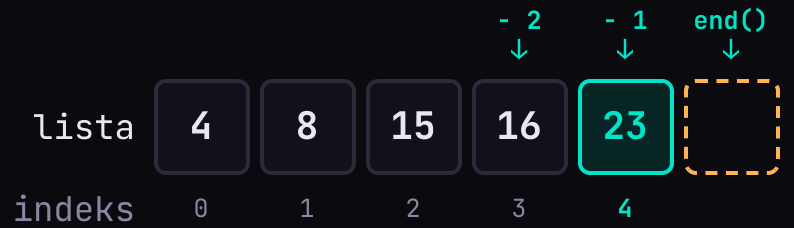
ŹLE — POD END() NIC NIE MA

```
cout << *lista.end();          // BŁĄD
```

DOBRZE — OSTATNI ELEMENT

```
cout << *(lista.end() - 1);    // 23
```

Iterator	pokazuje na
`lista.end()`	puste miejsce za vectorem
`lista.end() - 1`	ostatni element — **23**
`lista.end() - 2`	przedostatni — **16**



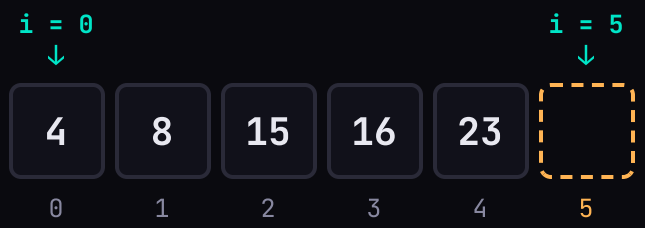
`*lista.end()` to tak jak `lista[lista.size()]` — czytanie spoza vectora. Program wypisze śmieci albo się wysypie.

08 Dlaczego end() jest ZA ostatnim? Powód 1

Bo tak samo działa **n** w zwykłej pętli po indeksach. Porównaj dwie pętle, które wypisują ten sam vector:

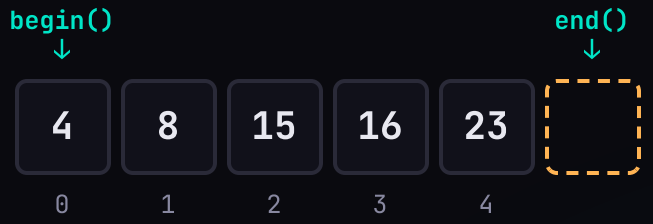
PĘTLA PO INDEKSACH

```
for(int i = 0; i < 5; i++){
    cout << lista[i] << ' ';
}
```



PĘTLA Z ITERATOREM

```
for(auto it = lista.begin(); it != lista.end(); it++){
    cout << *it << ' ';
}
```



Indeksu **5** też nie ma — to „pierwszy numer za vectorem”. Pętla kończy się, gdy i dojdzie do 5. Tak samo pętla z iteratorem kończy się, gdy strzałka dojdzie do **end()**. end() to dla iteratorów to, czym n jest dla indeksów.

08 Dlaczego end() jest ZA ostatnim? Powody 2 i 3

Powód 2: łatwo policzyć elementy

Od begin() do end() jest dokładnie tyle kroków, ile elementów: `lista.end() - lista.begin()` = **5** = `lista.size()`.



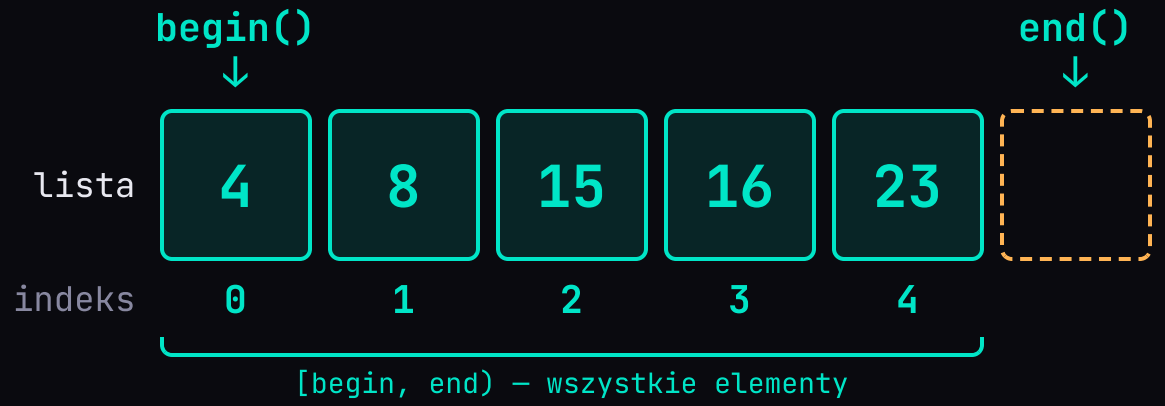
Powód 3: pusty vector

W pustym vectorze `begin() == end()` — obie strzałki stoją w tym samym miejscu, więc od razu widać, że nie ma elementów. Gdyby end() pokazywał ostatni element, w pustym vectorze nie miałby na co pokazywać!



08 Zakres [begin, end)

Od begin() — wliczamy · do end() — nie wliczamy



Taki zapis — nawias kwadratowy z lewej, okrągły z prawej — oznacza: „lewy koniec należy, prawy nie należy". Zakres [begin, end) to dokładnie **wszystkie** elementy vectora. Ten sam pomysł co w pętli i < n: od 0 włącznie do n wyłącznie.

08 Pętla z iteratorem — krok po kroku

```
vector<int> lista = {4, 8, 15, 16, 23};
for(auto it = lista.begin(); it != lista.end(); it++){
    cout << *it << ' ';
}
```

Obrót	it pokazuje na	it != end() ?	wypisujemy *it	na ekranie
1	indeks 0	tak	**4**	`4`
2	indeks 1	tak	**8**	`4 8`
3	indeks 2	tak	**15**	`4 8 15`
4	indeks 3	tak	**16**	`4 8 15 16`
5	indeks 4	tak	**23**	`4 8 15 16 23`
—	end()	**nie**	koniec pętli	`4 8 15 16 23`

08 for-each to iteratory w przebraniu

TO, CO PISZEMY

```
for(auto el : lista){
    cout << el << ' ';
}
```

TO, CO C++ ROBI „POD SPODEM”

```
for(auto it = lista.begin(); it != lista.end(); it++){
    auto el = *it;
    cout << el << ' ';
}
```

Pętla for-each z rozdziału 06 sama w środku używa **begin()** i **end()**.

1. Startuje od strzałki **begin()**.
2. W każdym obrocie bierze do el to, co jest pod strzałką.
3. Przesuwa strzałkę o jeden i kończy, gdy dojdzie do **end()**.

Dlatego for-each sam wie, gdzie zacząć i gdzie skończyć — a my nie piszemy żadnego i.

08 Dlaczego używamy iteratorów?

Skoro mamy indeksy, po co nam strzałki? Oto **cztery** powody:

1. Metody vectora

`erase` i `insert` dostają **iterator**, a nie indeks — zobaczysz je za chwilę.

2. Algorytmy

Gotowe funkcje, np. `sort` czy `reverse`, dostają **dwie strzałki**: od którego elementu zacząć i przed którym skończyć.

3. Struktury bez indeksów

Zbiór (set) i mapa (map), które poznasz później, **nie mają** [ ]. Ale mają begin() i end() — iteratory działają wszędzie tak samo.

4. Pętla for-each

Działa tylko dlatego, że struktura ma begin() i end().

08 Podgląd: iteratory w algorytmach

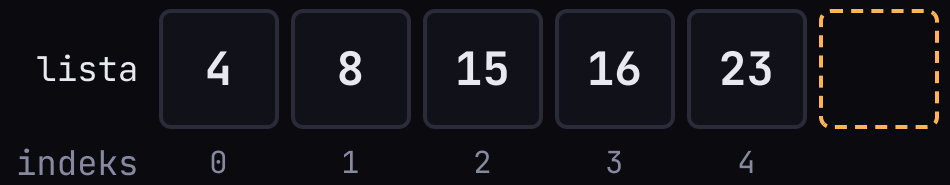
```cpp
#include <algorithm>   // tu mieszkają sort i reverse

vector<int> a = {5, 2, 9, 1, 7};
sort(a.begin(), a.end());          // {1, 2, 5, 7, 9}   — cały vector
vector<int> b = {5, 2, 9, 1, 7};
sort(b.begin(), b.begin() + 3);    // {2, 5, 9, 1, 7}   — tylko pierwsze 3
vector<int> r = {1, 2, 3, 4};
reverse(r.begin(), r.end());       // {4, 3, 2, 1}      — odwróć
```



Sortowanie poznasz dokładnie na następnej lekcji. Tu ważne jest tylko to, że funkcja dostaje **dwie strzałki** i działa na zakresie od pierwszej (wliczamy) do drugiej (nie wliczamy).

08 Sprawdź się: na co pokazuje strzałka?



Dla każdego zapisu powiedz, na który element pokazuje strzałka (albo ile wynosi wynik):

`lista.begin() + 2`
?

`lista.end() - 1`
?

`lista.begin() + 5`
?

`lista.end() - 5`
?

`*(lista.begin() + 1)`
?

`lista.end() - lista.begin()`
?

Zastanów się, **zanim** przejdziesz dalej.

08 Odpowiedzi



`lista.begin() + 2`

15 — indeks 2

`lista.end() - 1`

23 — ostatni

`lista.begin() + 5`

nic — to samo miejsce co end()

`lista.end() - 5`

4 — to samo co begin()

`*(lista.begin() + 1)`

8 — gwiazdka: wartość

`lista.end() - lista.begin()`

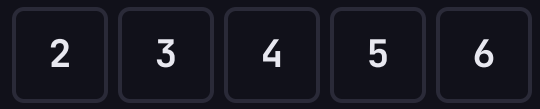
5 — liczba elementów

08 erase — usuwa jeden element

erase dostaje **iterator** — strzałkę na element, który ma zniknąć. Cztery **osobne** przykłady — za każdym razem startujemy od tej samej listy {1, 2, 3, 4, 5, 6}:

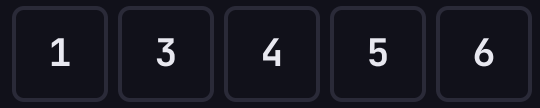
```
lista.erase(lista.begin());
```

usunie 1



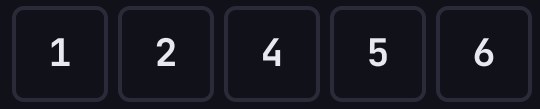

```
lista.erase(lista.begin() + 1);
```

usunie 2




```
lista.erase(lista.begin() + 2);
```

usunie 3




```
lista.erase(lista.end() - 1);
```

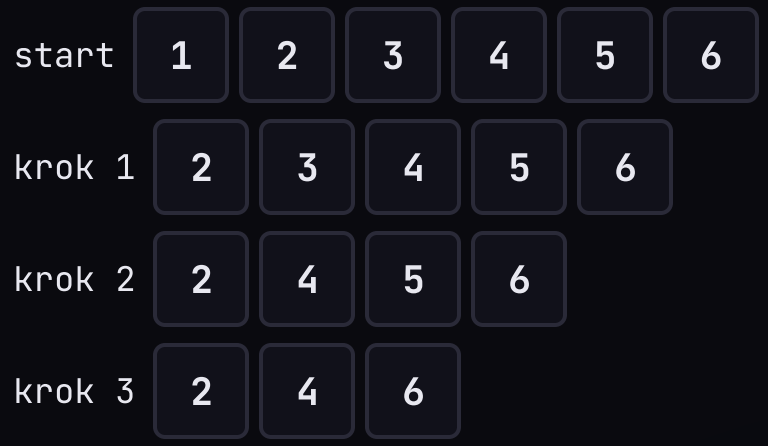
usunie 6 (ostatni)



> erase usuwa element z podanej pozycji, a wszystkie elementy za nim **przesuwają się w lewo**. size() maleje o 1.

08 Uwaga: erase po kolei przesuwa indeksy

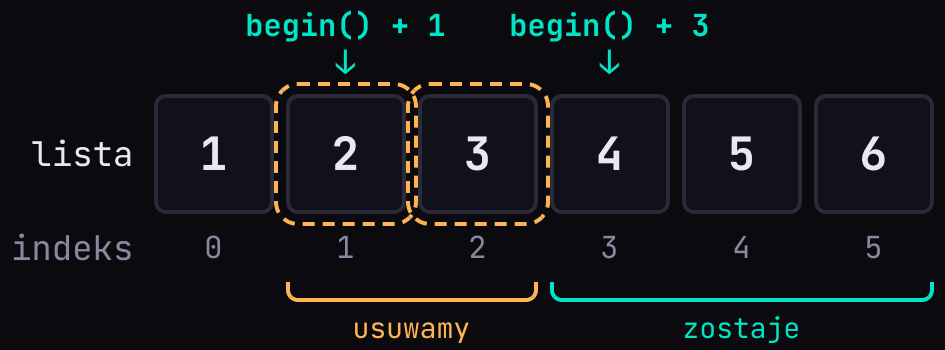
```
vector<int> lista = {1, 2, 3, 4, 5, 6};
lista.erase(lista.begin());       // usuwa 1
lista.erase(lista.begin() + 1);   // usuwa 3!
lista.erase(lista.begin() + 2);   // usuwa 5!
```

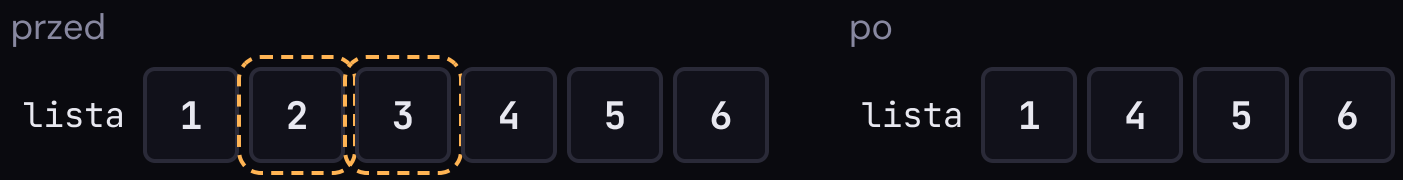


Uwaga: gdy wykonamy te trzy linijki **jedna po drugiej**, wynik jest inny niż w trzech osobnych przykładach. Po usunięciu 1 na indeksie 1 stoi już **3**, a nie 2. Wynik: **{2, 4, 6}**.

08 erase z dwiema strzałkami — zakres [od, do)

```
vector<int> lista = {1, 2, 3, 4, 5, 6};
lista.erase(lista.begin() + 1, lista.begin() + 3);   // {1, 4, 5, 6}
```

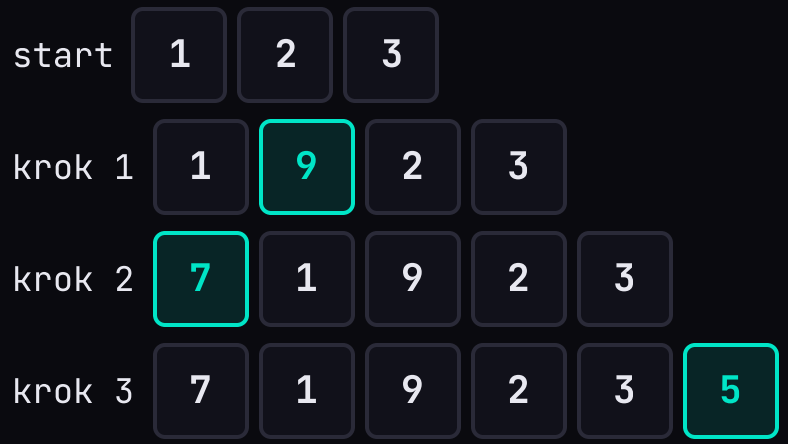




Gdy podamy **dwie** strzałki, erase usuwa cały kawałek: od pierwszej strzałki (**wliczamy**) do drugiej (**nie wliczamy**). Dokładnie tak jak begin() i end(): usuwamy indeksy 1 i 2, a 3 zostaje.

08 insert — wstawia element w środek

```
vector<int> lista = {1, 2, 3};
lista.insert(lista.begin() + 1, 9);   // {1, 9, 2, 3}
lista.insert(lista.begin(), 7);       // {7, 1, 9, 2, 3}
lista.insert(lista.end(), 5);         // {7, 1, 9, 2, 3, 5}
```



insert(iterator, wartość) wstawia element **przed** miejscem, na które wskazuje strzałka. Elementy za nim przesuwają się w prawo, a size() rośnie o 1. Wstawienie przed **end()** to wstawienie na sam koniec — tak jak push_back.

08 Metody vectora — ściąga

Metoda	Co robi	Przykład
`push_back(x)`	dodaje x na koniec	`{5} → {5, 7}`
`pop_back()`	usuwa ostatni element	`{4, 8, 15} → {4, 8}`
`size()`	zwraca liczbę elementów	`{1, 2, 5} → 3`
`clear()`	usuwa wszystkie elementy	`{1, 2} → {}`
`front() / back()`	pierwszy / ostatni element	`{4, 8, 15} → 4 / 15`
`assign(ile, co)`	wypełnia od nowa	`assign(3, 1) → {1, 1, 1}`
`resize(n)`	zmienia rozmiar (dokłada zera)	`{1} → resize(3) → {1, 0, 0}`
`begin() / end()`	strzałka na pierwszy / za ostatni	`*begin() → 1, *(end() - 1) → 3`
`erase(begin() + k)`	usuwa element o indeksie k	`{1, 2, 3}, k = 1 → {1, 3}`
`insert(begin() + k, x)`	wstawia x przed indeks k	`{1, 3}, k = 1, x = 2 → {1, 2, 3}`

09 Klasyczne wejście

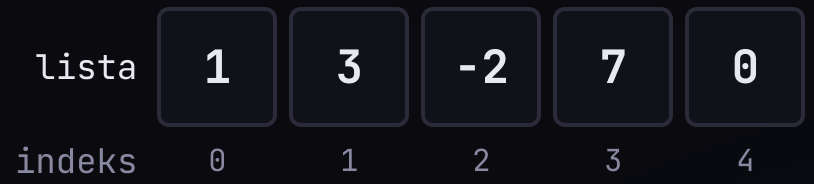
FORMAT (Z NOTATEK)

n
a_0 a_1 a_2 a_3 … a_{n-1}

PRZYKŁAD (Z NOTATEK)

5
1 3 -2 7 0

W pierwszej linii **n** — ile liczb. W drugiej — **n liczb** oddzielonych spacjami.



Notatki pokazują **pięć sposobów** wczytania takiego wejścia do vectora. Przejdziemy przez każdy — wszystkie dają ten sam wynik.

09 Sposób 1: vector(n) i pętla po indeksach

Z NOTATEK

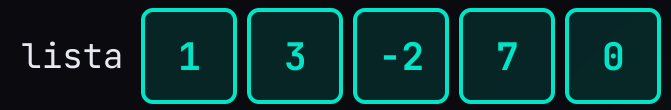
```
int n;
cin >> n;
vector<int> lista(n);
for(int i = 0; i < n; i++){
    cin >> lista[i];
}
```

1. Wczytujemy `n` = 5.
2. `vector<int> lista(n);` tworzy 5 zer — od razu są szufladki 0…4.
3. Pętla nadpisuje każde zero wczytaną liczbą: `cin >> lista[i];`

przed pętlą



po pętli



09 Sposób 2: vector(n) i for-each z &

Z NOTATEK

```
int n;
cin >> n;
vector<int> lista(n);
for(auto &e : lista){
    cin >> e;
}
```

1. Tak samo tworzymy n zer.
2. `for(auto &e : lista)` — e to **referencja** do kolejnej szufladki.
3. `cin >> e;` wpisuje liczbę **prosto do listy**.

Pułapka: bez `&` wczytywalibyśmy liczby do **kopii** — lista zostałaby pełna zer!

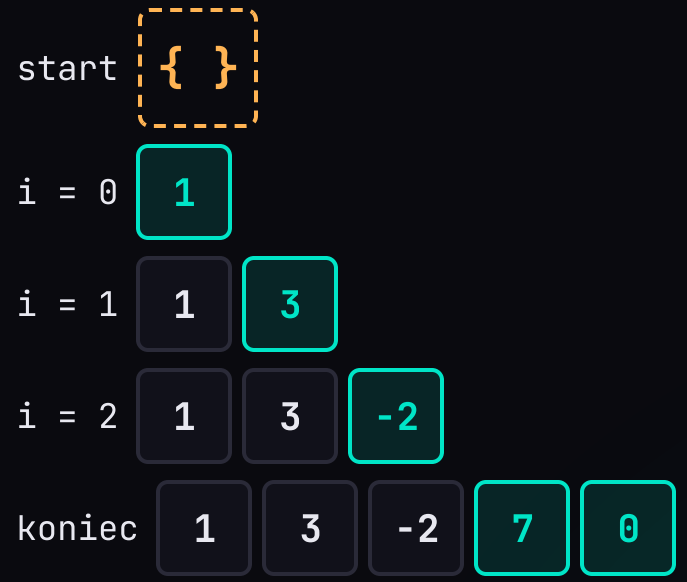
„99.9% olimpijczyków używa tych dwóch metod (ja używam tej drugiej)" — z notatek.

09 Sposób 3: pusty vector i push_back

Z NOTATEK

```
int n, a;
cin >> n;
vector<int> lista;
for(int i = 0; i < n; i++){
    cin >> a;
    lista.push_back(a);
}
```

Startujemy od **pustego** vectora. Każdą liczbę wczytujemy najpierw do zmiennej `a`, a potem dokładamy ją na koniec.



09 Sposób 4: while(n--) i push_back

Z NOTATEK

```
int n, a;
cin >> n;
vector<int> lista;
while(n--){
    cin >> a;
    lista.push_back(a);
}
```

`while(n--)` to krótki sposób na „powtórz n razy”. Sprawdza n, a **potem** zmniejsza je o 1:

sprawdzamy n	różne od 0?	n potem	obrót
5	tak	4	1.
4	tak	3	2.
3	tak	2	3.
2	tak	1	4.
1	tak	0	5.
0	**nie**	-1	koniec

Pętla wykonała się dokładnie **5 razy**. Uwaga: po pętli n nie jest już równe 5!

09 Sposób 5: `while(n--)` i indeks z `size()`

Z NOTATEK

```
int n;
cin >> n;
vector<int> lista(n);
while(n--){
    cin >> lista[lista.size() - n - 1];
}
```

Tworzymy n zer, a indeks liczymy ze wzoru **size() – n – 1**. Dla n = 5 (size() = 5):

n w środku pętli	indeks = 5 - n - 1
4	5 – 4 – 1 = **0**
3	5 – 3 – 1 = **1**
2	5 – 2 – 1 = **2**
1	5 – 1 – 1 = **3**
0	5 – 0 – 1 = **4**

Indeksy wychodzą po kolei: 0, 1, 2, 3, 4. Działa, ale łatwo się pomylić — na start wybierz sposób 1 albo 2.

09 Pięć sposobów — ten sam wynik

Sposób	Tworzenie	Pętla	Wpisanie
1	`vector<int> lista(n);`	`for(int i = 0; i < n; i++)`	`cin >> lista[i];`
2	`vector<int> lista(n);`	`for(auto &e : lista)`	`cin >> e;`
3	`vector<int> lista;`	`for(int i = 0; i < n; i++)`	`cin >> a;` `lista.push_back(a);`
4	`vector<int> lista;`	`while(n--)`	`cin >> a;` `lista.push_back(a);`
5	`vector<int> lista(n);`	`while(n--)`	`cin >> lista[lista.size()-n-1];`

Wynik: dla wejścia z notatek wszystkie dają {1, 3, −2, 7, 0}. Najczęściej używa się sposobu **1** i **2**.

09 Wczytaj i wypisz — cały program

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main(){
    int n;
    cin >> n;
    vector<int> lista(n);
    for(auto &e : lista){
        cin >> e;
    }
    for(auto el : lista){
        cout << el << ' ';
    }
    cout << endl;
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
5
1 3 -2 7 0
```

NA EKRANIE

```
1 3 -2 7 0
```

Wczytujemy **z &** (bo zmieniamy listę), wypisujemy **bez &** (bo tylko czytamy).

09 Wczytywanie innych typów

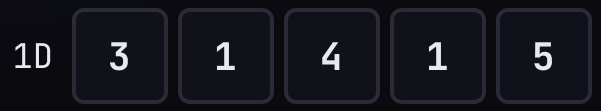
Zmienia się tylko **typ** w nawiasach ostrych. Reszta kodu zostaje taka sama:

Typ elementów	Tworzenie	Przykładowe wejście
liczby całkowite	`vector<int> lista(n);`	`5` / `1 3 15 3 8`
ułamki	`vector<double> lista(n);`	`4` / `1.14 5.33 8.12 7.29`
znaki	`vector<char> lista(n);`	`4` / `a c b d`
napisy	`vector<string> lista(n);`	`5` / `c++ programowanie kod oi oij`

Przy napisach dopisz na górze `#include <string>`. cin czyta napis do najbliższej spacji — jedno słowo trafia do jednej szufladki.

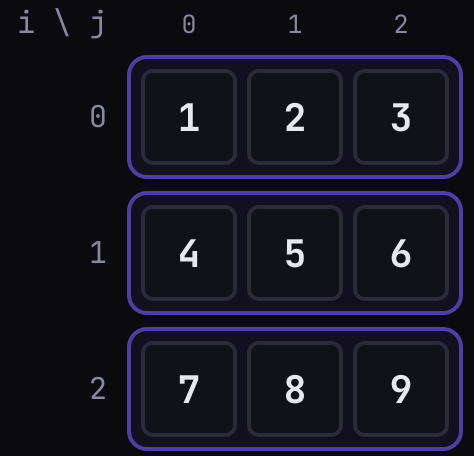
10 Vector 1D i vector 2D

vector 1D — jeden rząd szufladek:



`vector<int>`, `vector<char>`, `vector<string>`

vector 2D — tabela: wiersze i kolumny:



`vector<vector<int>>`, `vector<vector<char>>`, `vector<vector<string>>`

Vector 2D to **vector vectorów**: każdy element dużego vectora jest całym małym vectorem — jednym wierszem tabeli.

10 Vector vectorów — każdy wiersz to vector

```
vector<vector<int>> m = {{1, 2, 3}, {4, 5, 6}};
```

m

i \ j	0	1	2
0	1	2	3
1	4	5	6

1. Duży vector ma **2 elementy** — `m.size()` = 2 (liczba wierszy).
2. Każdy jego element to `vector<int>`: `{1, 2, 3}` oraz `{4, 5, 6}`.
3. Długość wiersza: `m[0].size()` = 3 (liczba kolumn).

10 Prostokąt 3 × 2 z zerami

```
vector<vector<int>> prostokat(3, vector<int>(2, 0));
                              ①  ②
```

① `3` — **ile wierszy**

② `vector<int>(2, 0)` — jak wygląda **jeden wiersz** — dwa zera

prostokat

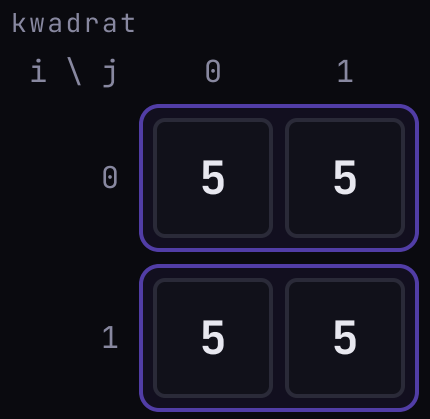
i \ j	0	1
0	0	0
1	0	0
2	0	0

To ten sam wzór co `vector<int> lista(5, 0);` — „ile razy, co”.
Tylko że teraz powtarzamy nie liczbę, ale **cały wiersz**.

10 Kwadrat 2 × 2 z piątkami

Z NOTATEK

```
vector<vector<int>> kwadrat(2, vector<int>(2, 5));
```

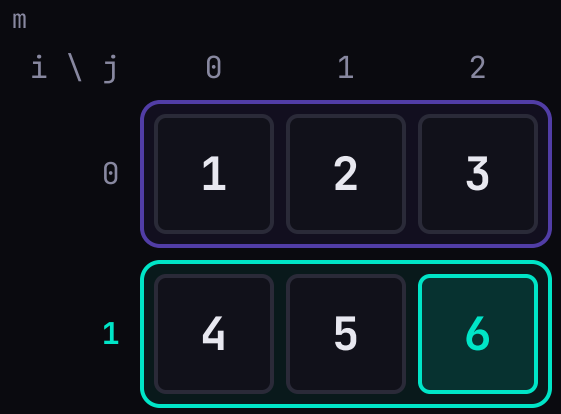


Zapis	Znaczenie
`2`	2 wiersze
`vector<int>(2, 5)`	każdy wiersz: dwie piątki

Pusty vector 2D: `vector<vector<int>> lista;` — nie ma w nim ani jednego wiersza.

10 Indeksowanie 2D: [wiersz][kolumna]

```
vector<vector<int>> m = {{1, 2, 3}, {4, 5, 6}};
```



Zapis	Typ	Co to jest
`m`	`vector<vector<int>>`	cała tabela
`m[1]`	`vector<int>`	wiersz {4, 5, 6}
`m[1][2]`	`int`	liczba **6**

Z notatek: **vector<vector<int>> → indeksowanie → vector<int> → indeksowanie → int**. Pierwszy nawias wybiera **wiersz**, drugi — **kolumnę** w tym wierszu.

10 Wypisywanie 2D — wiersz po wierszu

Z NOTATEK

```
for(int i = 0; i < 3; i++){          // i = 0, 1, 2
    vector<int> tmp = prostokat[i];  // wiersz i
    for(int j = 0; j < 2; j++){
        cout << tmp[j] << ' ';       // kolumny
    }
    cout << endl;
}
```

Pętla zewnętrzna przechodzi po **wierszach**. `tmp` to kopia wiersza i — zwykły vector<int>. Pętla wewnętrzna wypisuje jego **kolumny**, a po wierszu — endl.

NA EKRANIE

```
0 0
0 0
0 0
```

10 To samo krócej: `prostokat[i][j]`

Z NOTATEK: TO JEST TO SAMO CO

```
for(int i = 0; i < 3; i++){
    for(int j = 0; j < 2; j++){
        cout << prostokat[i][j] << ' ';
    }
    cout << endl;
}
```

Zamiast robić kopię wiersza `tmp`, od razu piszemy dwa nawiasy: `prostokat[i][j]`.

Jeszcze lepiej: liczba wierszy to `prostokat.size()`, a liczba kolumn — `prostokat[i].size()`. Wtedy nie trzeba wpisywać 3 i 2 na sztywno.

10 Dwie pętle — w jakiej kolejności?

prostokat — pary (i, j)

i \ j	0	1
0	0,0	0,1
1	1,0	1,1
2	2,0	2,1

1. Pętla zewnętrzna ustawia **wiersz i**.
2. Pętla wewnętrzna przechodzi po **wszystkich kolumnach j** tego wiersza.
3. Kolejność: (0,0) (0,1) → nowa linia → (1,0) (1,1) → nowa linia → (2,0) (2,1).

10 Przykład z notatek: 5 × 5 zer i 3 × 3 jedynek

```cpp
vector<vector<int>> lista(5, vector<int>(5, 0));
cout << "lista: \n";
for(int i = 0; i < 5; i++){
    for(int j = 0; j < 5; j++){
        cout << lista[i][j] << ' ';
    }
    cout << endl;
}
vector<vector<int>> lista1(3, vector<int>(3, 1));
cout << "lista1: \n";
for(int i = 0; i < 3; i++){
    for(int j = 0; j < 3; j++){
        cout << lista1[i][j] << ' ';
    }
    cout << endl;
}
```

NA EKRANIE

```
lista:
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
lista1:
1 1 1
1 1 1
1 1 1
```

10 assign w 2D — tak samo jak konstruktor

Z NOTATEK

```
vector<vector<int>> lista;              // pusty 2D
lista.assign(5, vector<int>(2, 0));     // 5 wierszy {0, 0}
lista.assign(3, vector<int>(3, -1));    // 3 wiersze {-1, -1, -1}
lista.assign(1, vector<int>(4, 3));     // 1 wiersz {3, 3, 3, 3}
```

assign(5, …(2, 0))

i \ j	0	1
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

assign(3, …(3, -1))

i \ j	0	1	2
0	-1	-1	-1
1	-1	-1	-1
2	-1	-1	-1

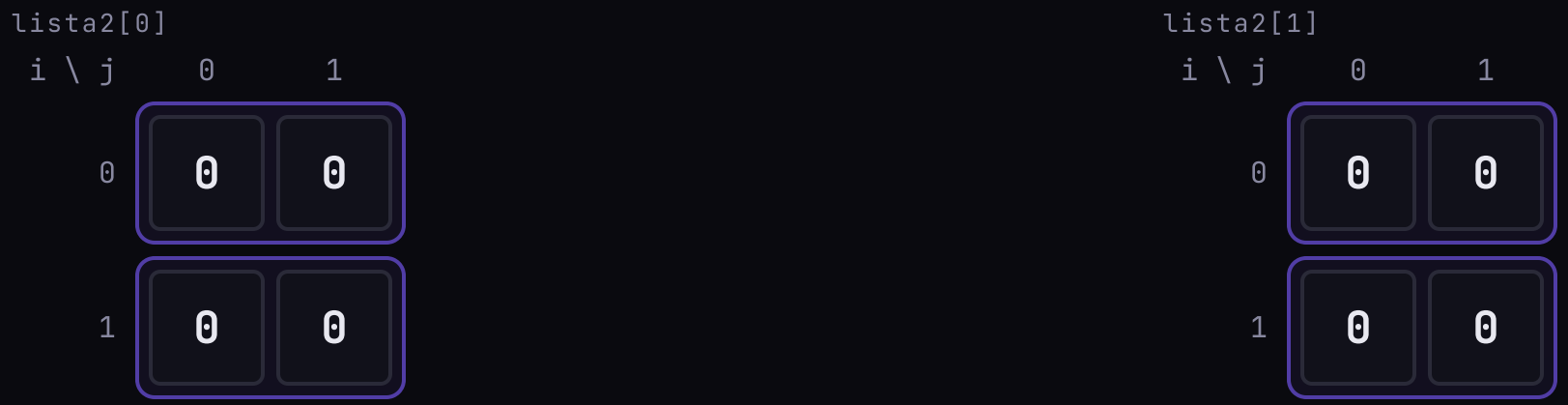
assign(1, …(4, 3))

i \ j	0	1	2	3
0	3	3	3	3

10 Dla ciekawych: vector 3D

Z NOTATEK

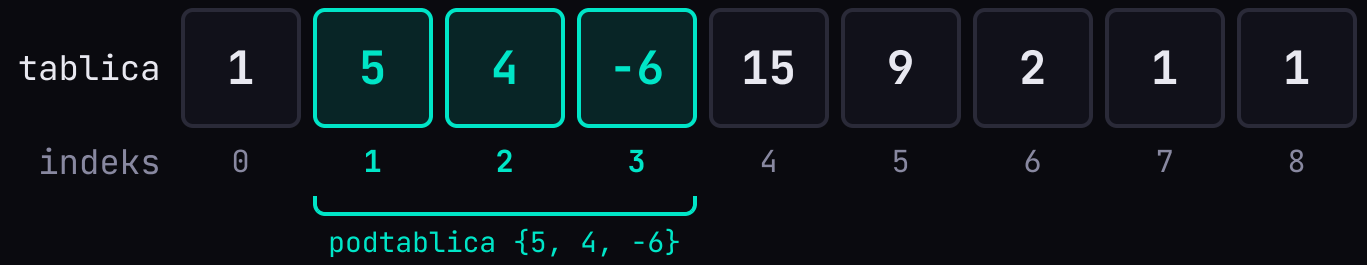
```
vector<vector<vector<int>>> lista2(2, vector<vector<int>>(2, vector<int>(2, 0)));
```



To vector, którego elementami są vectory 2D. Tutaj: 2 „warstwy”, w każdej 2 wiersze po 2 kolumny. Dostęp: `lista2[i][j][k]` — warstwa, wiersz, kolumna.

11 Podtablica (subarray) — definicja

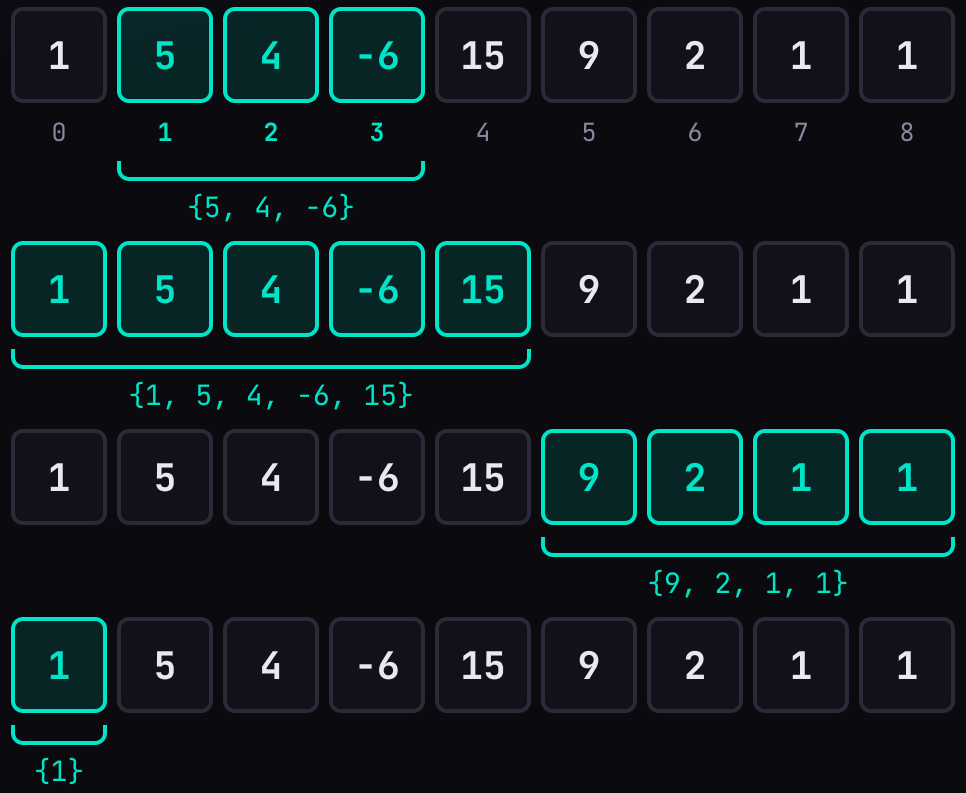
Podtablica to kawałek tablicy z elementów stojących obok siebie, bez przerw.



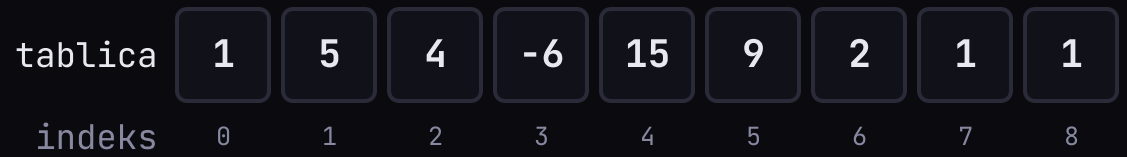
Wybieramy jakiś początek i jakiś koniec — i bierzemy **wszystko pomiędzy**. Nie wolno niczego pominąć ani zmieniać kolejności. Po angielsku: **subarray**. W vectorze działa to dokładnie tak samo.

11 Przykłady z notatek

Z tablicy {1, 5, 4, −6, 15, 9, 2, 1, 1} wycinamy cztery różne podtablice:



11 Czy to jest podtablica?



{5, 4, -6}

TAK — indeksy 1, 2, 3 stoją obok siebie

{4, 15}

NIE — pomija -6, jest przerwa

{9}

TAK — jeden element też jest podtablicą

cała tablica

TAK — od pierwszego do ostatniego

{-6, 4}

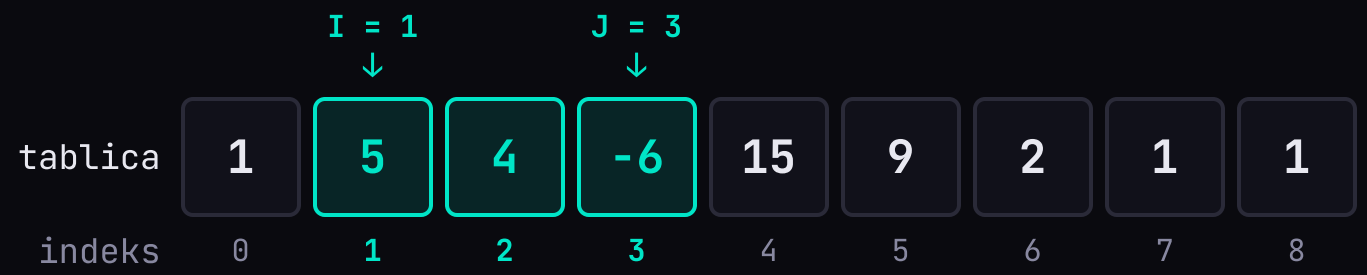
NIE — zła kolejność

{2, 1, 1}

TAK — indeksy 6, 7, 8

11 Początek I i koniec J

Każda podtablica ma jakąś pozycję początkową (nazwijmy ją **I**) **oraz jakąś pozycję końcową** (**J**). Załóżmy, że tablica ma rozmiar **N**.



WTEDY: $0 \le I, J < N$ oraz $I \le J$

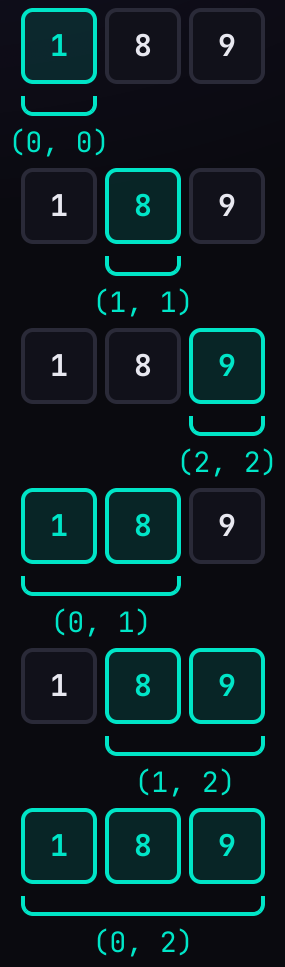
I i J to indeksy, więc są od 0 do N−1. Początek nie może być za końcem: I ≤ J. Gdy I = J, podtablica ma jeden element.

11 Podtablica = para (I, J)

Podtablica	I (początek)	J (koniec)	długość J - I + 1
{5, 4, -6}	1	3	3
{1, 5, 4, -6, 15}	0	4	5
{9, 2, 1, 1}	5	8	4
{1}	0	0	1

Żeby opisać podtablicę, wystarczą **dwie liczby**: początek i koniec. To idealne miejsce na **parę** z lekcji 9: `pair<int, int> p = {1, 3};` — para ‹początek; koniec›.

11 Wszystkie podtablice tablicy {1, 8, 9}



podtablica	para <początek; koniec>
1	0 0
8	1 1
9	2 2
1 8	0 1
8 9	1 2
1 8 9	0 2

N = 3, więc 0 ≤ I, J < 3. Aby wygenerować **wszystkie podtablice**, trzeba umieć wygenerować **wszystkie pary (I, J)** z I ≤ J.

11 Ile jest podtablic?

Policzmy, ile podtablic zaczyna się na każdym indeksie — dla tablicy o długości **N = 4**:

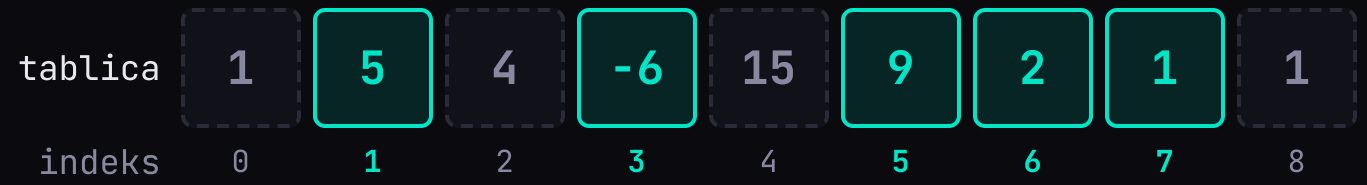
początek I	możliwe końce J	ile
0	0, 1, 2, 3	4
1	1, 2, 3	3
2	2, 3	2
3	3	1

Wzór: razem 4 + 3 + 2 + 1 = **10**. Ogólnie: **N · (N + 1) / 2**. Dla N = 3: 3 · 4 / 2 = 6 — zgadza się!

N	1	2	3	4	5	10	100
podtablic	1	3	6	10	15	55	5050

12 Podzbiór (subset) — definicja

Podzbiór to dowolne elementy tablicy — nie muszą stać obok siebie.



Dla każdego elementu decydujemy: **bierzemy** albo **nie bierzemy**. Wybrane elementy zostają w tej samej kolejności co w tablicy. Po angielsku: **subset**. Powyżej: podzbiór {5, -6, 9, 2, 1}.

12 Przykłady z notatek

Trzy podzbiory tej samej tablicy (podświetlone = wybrane, szare = pominięte):



{5, -6, 9, 2, 1} — z przerwami

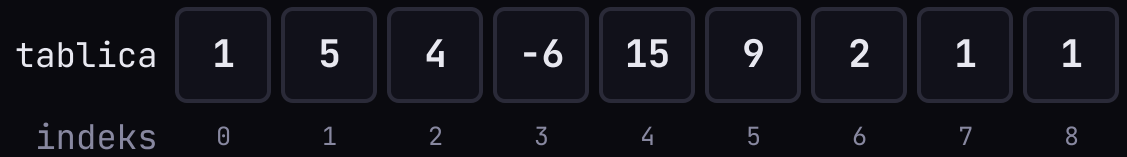


{4, -6, 15} — bez przerw (to jednocześnie podtablica!)



{5, 4, -6, 2, 1} — kawałek, przerwa, kawałek

12 Czy to jest podzbiór?



{1, 15, 1}
TAK — np. indeksy 0, 4, 7

{5, 4}
TAK — i to nawet podtablica

{1, 1, 1}
TAK — w tablicy są trzy jedynki (0, 7, 8)

{7}
NIE — w tablicy nie ma liczby 7

{9, 15}
NIE — 15 stoi przed 9, kolejność musi zostać

cała tablica
TAK — bierzemy wszystko

12 Wybór: bierzemy albo nie

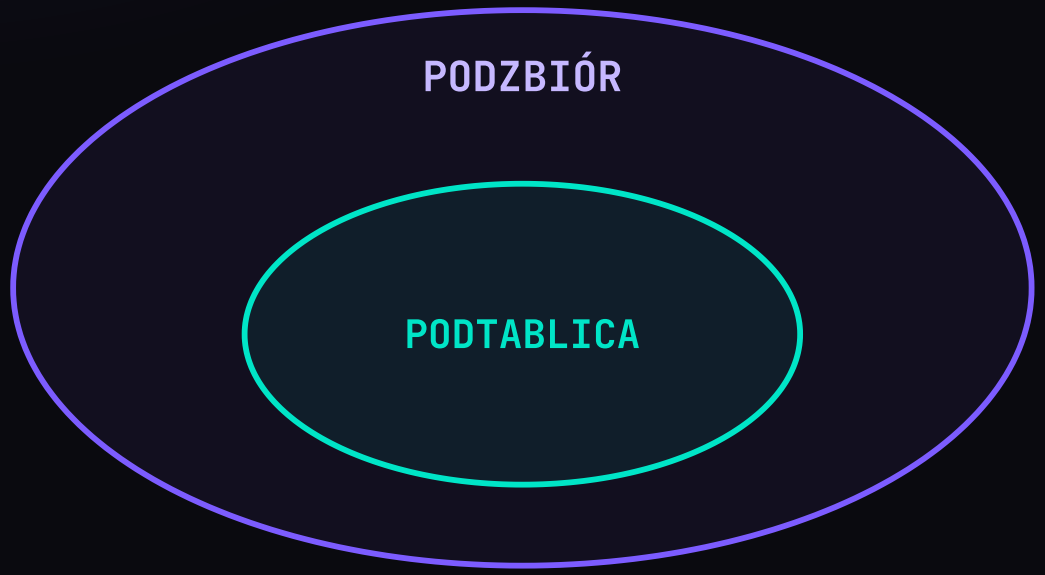
Tablica {1, 8, 9}. Przy każdym elemencie mówimy **TAK** albo **NIE**. Wszystkie możliwości:

1	8	9	podzbiór
nie	nie	nie	{ } — pusty
nie	nie	**tak**	{9}
nie	**tak**	nie	{8}
nie	**tak**	**tak**	{8, 9}
tak	nie	nie	{1}
tak	nie	**tak**	{1, 9}
tak	**tak**	nie	{1, 8}
tak	**tak**	**tak**	{1, 8, 9}

Liczymy: każdy z N elementów ma **2** możliwości → $2 \cdot 2 \cdot 2 = 2^N$ = **8** (razem z pustym).

Niepustych podzbiorów jest $2^N - 1$ = **7**. Porównaj: podtablic tej tablicy było tylko **6** — brakuje {1, 9}.

12 Podtablica a podzbiór



rysunek z notatek

1. Każda podtablica **jest** podzbiorem — to po prostu podzbiór bez przerw.
2. Ale **nie każdy** podzbiór jest podtablicą.
3. Przykład: {1, 9} z tablicy {1, 8, 9} — podzbiór, ale nie podtablica (pomija 8).



12 Podtablic jest mało, podzbiorów — bardzo dużo

N	podtablice: N · (N + 1) / 2	podzbiory niepuste: $2^N - 1$
3	6	7
4	10	15
5	15	31
10	55	1 023
20	210	1 048 575
30	465	ponad miliard

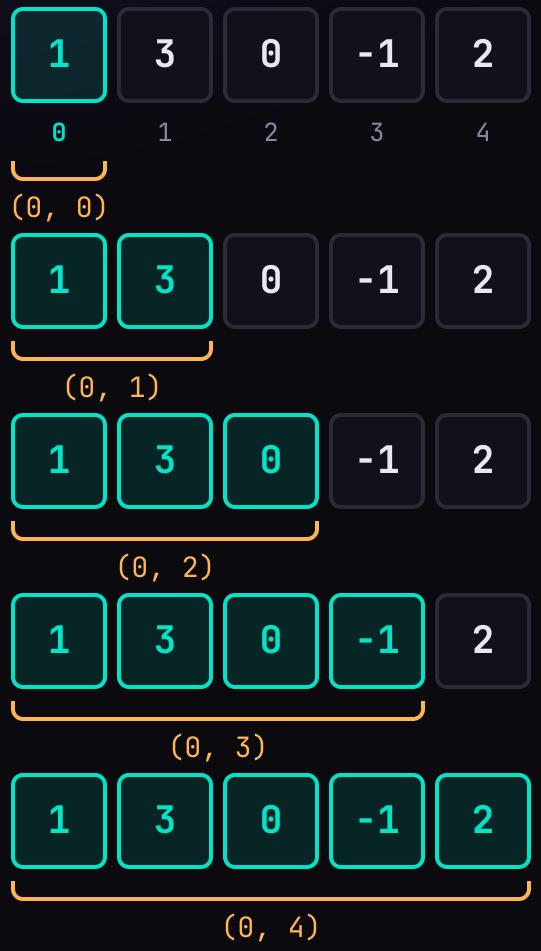
Podzbiorów szybko robi się bardzo dużo — dla N = 30 nie da się ich wszystkich wypisać. Podtablic jest „tylko” około $N^2 / 2$.

13 Prefiks — początek tablicy

Prefiks to podtablica, która **zaczyna się na samym początku** — na indeksie 0. Przykłady z notatek dla `vector<int>` {5, 4, 9, 1, 3, -5, 2, 1, 1, 7}:



13 Wszystkie prefiksy tablicy {1, 3, 0, −1, 2}



prefiks	I	J
{1}	0	0
{1, 3}	0	1
{1, 3, 0}	0	2
{1, 3, 0, −1}	0	3
{1, 3, 0, −1, 2}	0	4

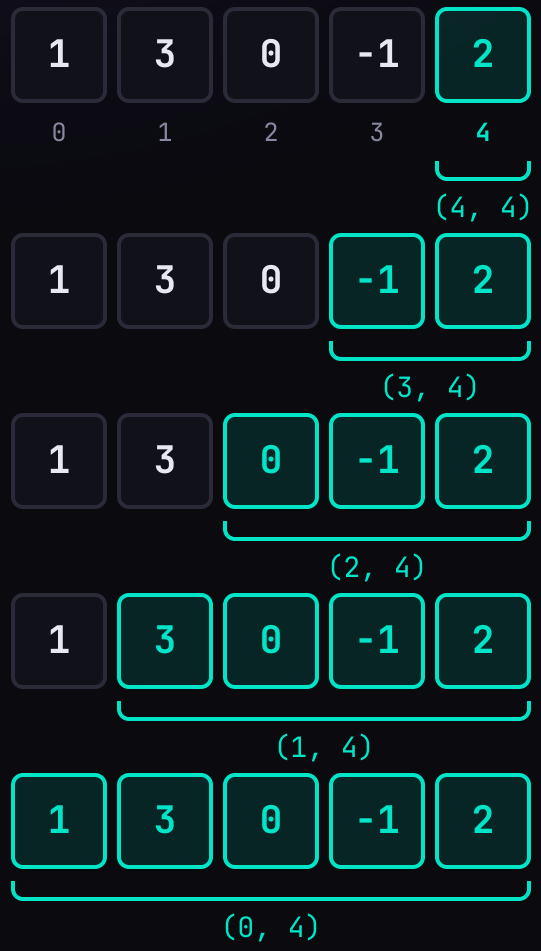
Tablica o długości n ma dokładnie **n prefiksów**. Początek zawsze I = 0, a koniec J = 0, 1, …, n−1.

13 Sufiks — koniec tablicy

Sufiks to podtablica, która **kończy się na samym końcu** — na indeksie n−1. Przykłady z notatek dla tej samej tablicy:



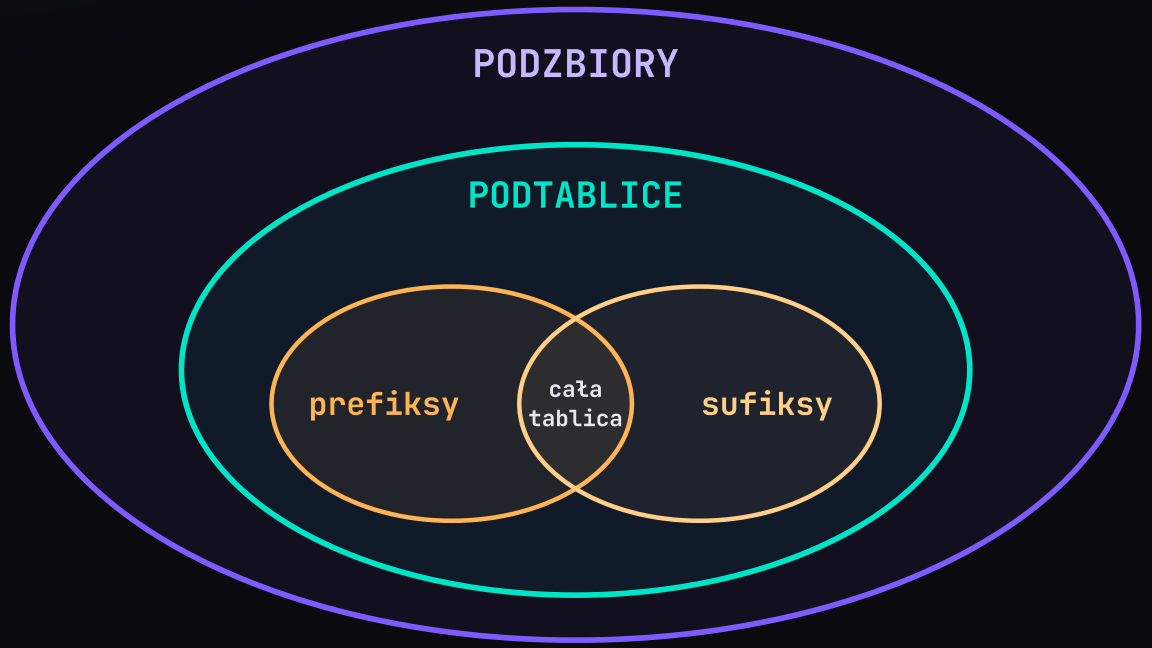
13 Wszystkie sufiksy tablicy {1, 3, 0, −1, 2}



sufiks	I	J
{2}	4	4
{−1, 2}	3	4
{0, −1, 2}	2	4
{3, 0, −1, 2}	1	4
{1, 3, 0, −1, 2}	0	4

Tablica o długości n ma dokładnie **n sufiksów**. Koniec zawsze J = n−1, a początek I = n−1, …, 1, 0.

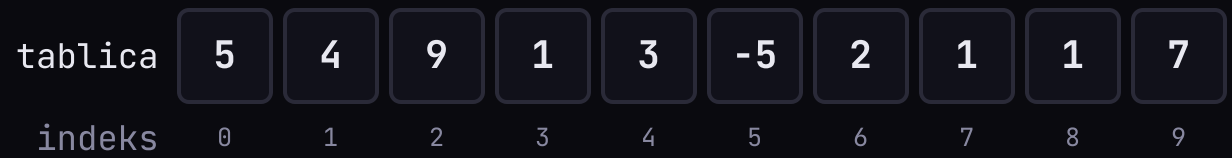
13 Prefiks, sufiks, podtablica, podzbiór



Pojęcie	Para (I, J)
podtablica	dowolne I ≤ J
prefiks	(0, J)
sufiks	(I, n−1)
cała tablica	(0, n−1) — prefiks i sufiks naraz

> Każdy prefiks i każdy sufiks jest podtablicą, a każda podtablica — podzbiorem.

13 Prefiks czy sufiks?



{5, 4}
prefiks — (0, 1)

{1, 7}
sufiks — (8, 9)

{9, 1, 3}
ani to, ani to — zwykła podtablica (2, 4)

cała tablica
i prefiks, i sufiks — (0, 9)

{5}
prefiks — (0, 0)

{7}
sufiks — (9, 9)

14 Zadania — od najłatwiejszych

Wszystkie zadania z notatek, po kolei. Najpierw tworzenie i wczytywanie, potem logika, a na końcu zadania, w których przydadzą się podtablice i prefiksy.

Zadania 1–7
tworzenie vectora

Zadania 8–11
wczytywanie do vectora

Logika 1–4
parzyste, największa, najmniejsza

Zadanie 1
pierwszy i ostatni (CodeChef)

Zadanie 2
Codeforces

Podtablice i prefiksy
3 zadania z rozwiązaniem krok po kroku

14 Zadania 1–7: tworzenie vectora

1. Stwórz wektor z wartościami **[1, 5, 7, 8]**.
2. Stwórz wektor z wartościami **[c, d, e, f]**.
3. Stwórz wektor z wartościami **[c++, program, olimpiada]**.
4. Stwórz wektor z wartościami **[3.14, 2.78, 1.25]**.
5. Stwórz wektor z **sześcioma zerami**.
6. Stwórz wektor ze **stoma piątkami**.
7. Stwórz wektor z **dwudziestoma napisami „C++”**.

Podpowiedź do 1–4: jaki to typ? Liczby całkowite → int, pojedyncze litery → char, słowa → string, ułamki → double. Wartości wpisz w klamrach (sposób 2).

Podpowiedź do 5–7: „n razy to samo” → nawiasy okrągłe (sposób 3): `vector<int> a(ile, co);`

14 Zadania 8–11: wczytaj input i zapisz listę do wektora

Nr	Format	Przykładowy input	Typ
8	n, potem $a_0 \dots a_{n-1}$	`5` `1 3 15 3 8`	liczby całkowite
9	n, potem $f_0 \dots f_{n-1}$	`4` `1.14 5.33 8.12 7.29`	ułamki
10	n, potem $c_0 \dots c_{n-1}$	`4` `a c b d`	znaki
11	n, potem $s_0 \dots s_{n-1}$	`5` `c++ programowanie kod oi oij`	napisy

Wybierz typ vectora i jeden ze sposobów wczytywania z rozdziału 09. Na koniec wypisz vector, żeby sprawdzić, czy wszystko się wczytało.

14 Zadania z logiką 1–4

1. Ile jest liczb **parzystych**?
2. Czego jest więcej: parzystych czy nieparzystych? Wypisz **„PARZYSTE”**, **„NIEPARZYSTE”** albo **„REMIS”**.
3. **Największa** liczba w liście.
4. **Najmniejsza** liczba w liście.

PRZYKŁADOWY INPUT: N I LISTA

```
5
1 3 -2 7 0
```

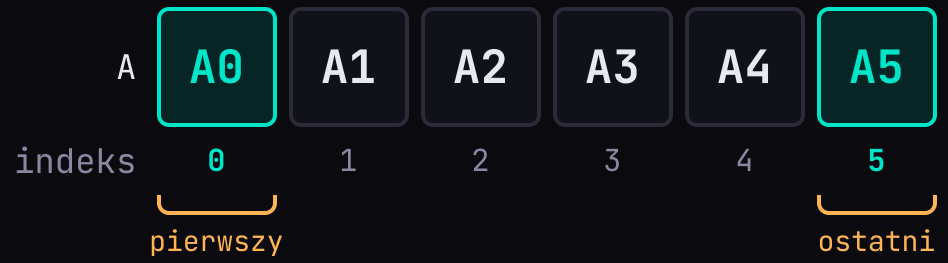
Zadanie	Wynik dla przykładu
1	2 (to −2 i 0)
2	NIEPARZYSTE (3 > 2)
3	7
4	−2

Gdzie szuka się największej wartości? Z notatek: najwyższa góra, lepszy z dwóch skoczków, największy rozmiar telewizora.

14 Zadanie 1: pierwszy i ostatni (CodeChef)

Mamy tablicę A o długości N. Możemy ją dowolnie wiele razy **obracać** — przesuwać elementy w kółko, tak że ostatni wskakuje na początek. Jaka jest **największa** możliwa suma **pierwszego i ostatniego** elementu?

Pełna treść i format wejścia: **vjudge.net/problem/CodeChef-FIRSTANDLAST**



Zanim zajrzysz dalej: spróbuj sam: wypisz na kartce wszystkie obroty tablicy z 6 elementami i zaznacz pierwszy oraz ostatni element w każdym.

14 Zadanie 1 — wskazówka z notatek

obrót 0	A0	A1	A2	A3	A4	A5
obrót 1	A5	A0	A1	A2	A3	A4
obrót 2	A4	A5	A0	A1	A2	A3
obrót 3	A3	A4	A5	A0	A1	A2
obrót 4	A2	A3	A4	A5	A0	A1
obrót 5	A1	A2	A3	A4	A5	A0

obrót	pierwszy + ostatni
0	A0 + A5
1	A5 + A4
2	A4 + A3
3	A3 + A2
4	A2 + A1
5	A1 + A0

Wniosek: pierwszy i ostatni to zawsze **dwaj sąsiedzi w kółku**: (A0, A5), (A5, A4), (A4, A3)… Wystarczy sprawdzić **każdą parę sąsiadów** — razem z parą A[N-1] i A[0].

14 Zadanie 1 — rozwiązanie (zajrzyj po swojej próbie)

```cpp
int n;
cin >> n;
vector<int> a(n);
for(auto &e : a){
    cin >> e;
}
int best = a[n - 1] + a[0];       // para „przez koniec”
for(int i = 0; i + 1 < n; i++){
    if(a[i] + a[i + 1] > best){   // para sąsiadów
        best = a[i] + a[i + 1];
    }
}
cout << best << endl;
```

1. Wczytujemy tablicę (sposób 2).
2. Na start `best` = suma pary „przez koniec”: ostatni + pierwszy.
3. Sprawdzamy wszystkie pary sąsiadów i zapamiętujemy największą sumę.

Uwaga: sprawdź w treści dokładny format wejścia — np. czy w jednym pliku jest kilka testów.

14 Zadanie 2 (Codeforces)

Treść zadania znajdziesz pod linkiem z notatek (zadanie w grupie na Codeforces — może być potrzebne logowanie):

codeforces.com/group/MWSDmqGsZm/contest/219774/problem/L

Zanim zaczniesz: upewnij się, że rozumiesz **podtablice** i **podzbiory** (rozdziały 11 i 12) — w notatkach definicje stoją zaraz przy tym zadaniu.

14 Zadanie: wypisz wszystkie podtablice

INPUT

n

a_0 a_1 a_2 … a_{n-1}

PRZYKŁADOWY INPUT

```
3
1 8 9
```

PRZYKŁADOWY OUTPUT (Z NOTATEK)

```
1
8
9
1 8
8 9
1 8 9
```

Każda linijka to jedna podtablica. Najważniejsze: wypisać **wszystkie** i każdą **raz**. Rozwiążemy to w trzech krokach.

14 Krok 1: wszystkie pary (i, j)

Z NOTATEK

```
int n = 4;
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        cout << "{" << i << "," << j << "}" << endl;
    }
}
```

NA EKRANIE: 16 PAR

```
{0,0}
{0,1}
{0,2}
{0,3}
{1,0}
…
{3,3}
```

Uwaga: pary takie jak {1,0} mają i > j — koniec przed początkiem. To **nie są** podtablice!

14 Krok 2: elementy od i do j

Z NOTATEK

```
vector<int> lista = {1, 2, 5, 21};
int n = 4;
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        // wyświetlamy elementy od startu (i) do końca (j)
        for(int k = i; k <= j; k++){
            cout << lista[k] << ' ';
        }
        cout << endl;
    }
}
```

Trzecia pętla (k) idzie od początku i do końca j i wypisuje te elementy — to jest jedna podtablica.

> **Problem:** gdy i > j, pętla po k nie wykona się ani razu — na ekranie pojawi się **pusta linijka**. Pary z i > j trzeba pominąć.

14 Krok 3: j zaczyna się od i

Z NOTATEK — POPRAWIONE

```
vector<int> lista = {1, 2, 5, 21};
int n = 4;
for(int i = 0; i < n; i++){
    for(int j = i; j < n; j++){
        for(int k = i; k <= j; k++){
            cout << lista[k] << ' ';
        }
        cout << endl;
    }
}
```

NA EKRANIE

```
1
1 2
1 2 5
1 2 5 21
2
2 5
2 5 21
5
5 21
21
```

j startuje od i, więc zawsze i ≤ j. Wypisaliśmy wszystkie **10** podtablic (4 · 5 / 2 = 10) — każdą raz.

14 Kolejność wypisywania

Dla przykładu {1, 8, 9} nasz program wypisze podtablice **w innej kolejności** niż w notatkach:

NASZ PROGRAM — WEDŁUG POCZĄTKU

```
1
1 8
1 8 9
8
8 9
9
```

NOTATKI — WEDŁUG DŁUGOŚCI

```
1
8
9
1 8
8 9
1 8 9
```

To **te same** 6 podtablic. Jeśli zadanie wymaga konkretnej kolejności, trzeba zmienić pętle — np. najpierw długość, potem początek.

14 Zadanie: pary wszystkich prefiksów

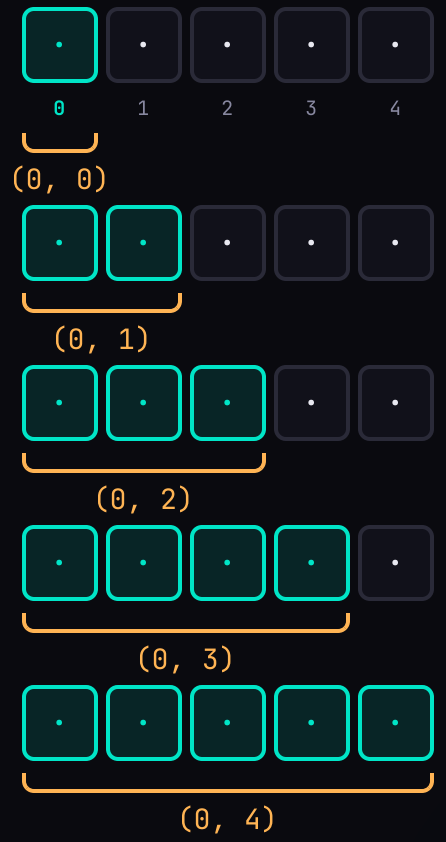
Wygeneruj i wyświetl wszystkie pary początków i końców wszystkich możliwych prefiksów tablicy o długości n.

PRZYKŁADOWY INPUT: N – ROZMIAR TABLICY

```
5
```

PRZYKŁADOWY OUTPUT

```
0 0
0 1
0 2
0 3
0 4
```



14 Pary prefiksów — rozwiązanie

```
int n;
cin >> n;
for(int j = 0; j < n; j++){
    cout << 0 << " " << j << endl;
}
```

Prefiks **zawsze zaczyna się na 0**. Zmienia się tylko koniec J: 0, 1, …, n−1. Wystarczy więc **jedna** pętla.

NA EKRANIE

```
0 0
0 1
0 2
0 3
0 4
```

14 Zadanie: suma na każdym prefiksie

Wyświetl sumę elementów na każdym prefiksie.

INPUT

n
a_0 a_1 … a_{n-1}

PRZYKŁADOWY INPUT

```
5
1 3 0 -1 2
```

PRZYKŁADOWY OUTPUT

```
1
4
4
3
5
```



14 Suma prefiksów — pomysł

Nie trzeba liczyć każdej sumy od nowa! Każdy kolejny prefiks to **poprzedni prefiks + jeden nowy element**:

prefiks	liczymy	suma
{1}	0 + 1	**1**
{1, 3}	1 + 3	**4**
{1, 3, 0}	4 + 0	**4**
{1, 3, 0, −1}	4 + (−1)	**3**
{1, 3, 0, −1, 2}	3 + 2	**5**

Trzymamy w zmiennej **suma** wynik dla poprzedniego prefiksu i w każdym kroku dodajemy kolejny element.

14 Suma prefiksów — rozwiązanie

```cpp
int n;
cin >> n;
vector<int> lista(n);
for(auto &e : lista){
    cin >> e;
}
int suma = 0;
for(int i = 0; i < n; i++){
    suma += lista[i];       // dokładamy nowy element
    cout << suma << endl;   // suma prefiksu (0, i)
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
5
1 3 0 -1 2
```

NA EKRANIE

```
1
4
4
3
5
```

15 Najczęstsze błędy (1/2)

01 **Indeks w pustym vectorze.** `vector<int> lista; lista[0] = 5;` — szufladki 0 nie ma. Użyj `push_back` albo `vector<int> lista(n);`.

02 **Nawiasy.** `vector<int> a(5, 0);` to pięć zer, a `vector<int> a = {5, 0};` — dwa elementy: 5 i 0.

03 `cout << lista;` — błąd kompilacji. Wypisuj pętlą.

04 **Brak & przy wczytywaniu.** `for(auto e : lista) cin >> e;` — liczby trafiają do kopii, lista zostaje pełna zer.

15 Najczęstsze błędy (2/2)

01 **Indeks poza zakresem.** `lista[lista.size()]` nie istnieje — ostatni element to `lista[lista.size() - 1]` albo `lista.back()`.

02 **erase po kolei.** Po usunięciu elementy przesuwają się w lewo — indeksy pozostałych elementów się zmieniają.

03 **end() to nie ostatni element.** `*lista.end()` czyta spoza vectora. Ostatni element to `*(lista.end() - 1)`. Iteratora nie wypisujemy bez gwiazdki: `cout << it;` — błąd.

04 **Brak `#include <vector>`.** Kompilator nie wie, co to vector.

05 **Kolejność w 2D.** `m[wiersz][kolumna]`. `vector<vector<int>> m(3, vector<int>(2));` to 3 wiersze po 2 kolumny — nie odwrotnie.

16 Ściąga: tworzenie i metody

Pusty

`vector<int> lista;` → `{}`

Z wartościami

`vector<int> lista = {1, 2, 3};`

n razy x / n zer

`vector<int> lista(5, 0);` · `lista(5)`

Odczyt / zmiana

`lista[2]` · `lista[2] = 7;`

Rozmiar

`lista.size()`

Dodaj / usuń z końca

`push_back(x)` · `pop_back()`

Pierwszy / ostatni

`front()` · `back()`

Wyczyść / wypełnij / zmień rozmiar

`clear()` · `assign(n, x)` · `resize(n)`

Iteratory

`begin()` → pierwszy · `end()` → za ostatnim · `*it` → wartość

Usuń / wstaw

`erase(begin() + k)` · `insert(begin() + k, x)`

16 Ściąga: wczytywanie, wypisywanie, 2D

WCZYTAJ N LICZB

```
int n;
cin >> n;
vector<int> lista(n);
for(auto &e : lista){
    cin >> e;
}
```

WYPISZ

```
for(auto el : lista){
    cout << el << ' ';
}
cout << endl;
```

2D: 3 WIERSZE PO 2 ZERA

```
vector<vector<int>> m(3, vector<int>(2, 0));
```

WYPISZ 2D

```
for(int i = 0; i < m.size(); i++){
    for(int j = 0; j < m[i].size(); j++){
        cout << m[i][j] << ' ';
    }
    cout << endl;
}
```

16 Ściąga: podtablica, podzbiór, prefiks, sufiks

Pojęcie	Co to jest	Para (I, J)	Ile dla n
podtablica	elementy obok siebie, bez przerw	$I \le J$	$n \cdot (n+1) / 2$
podzbiór	dowolne elementy, kolejność zostaje	—	$2^n - 1$ niepustych
prefiks	podtablica od początku	(0, J)	n
sufiks	podtablica do końca	(I, n−1)	n

Każdy prefiks i sufiks jest podtablicą, a każda podtablica — podzbiorem.

// KONIEC LEKCJI 10

Umiesz już korzystać z vectora!

Tworzenie, indeksowanie, metody, wypisywanie, for-each z referencją, iteratory begin() i end(), wczytywanie, vector 2D, podtablice, podzbiory, prefiksy i sufiksy. Teraz zadania! Na następnej lekcji: **sortowanie** — iteratory przydadzą się od razu: sort(lista.begin(), lista.end()).

